GUIA COMPLETO DO PROJETO

Controle de Produção

Tudo sobre o sistema: o que ele faz, o que foi construído, como o código está organizado, como conectar cada serviço externo (passo a passo, com o motivo de cada passo) e o que ainda está em aberto.

Escrito para quem é dono do produto e não precisa ler código para acompanhar.

Versão de 26/10/2026 · branch `claude/prompt-mestre-producao-s2y4fq` · commit `a079f7e`
Este documento não contém nenhuma senha, chave ou segredo.

Sumário

1. Como usar este guia
2. O projeto em uma página
3. O que o sistema faz
4. Como as peças se encaixam
5. O que foi feito, em ordem
6. As regras do projeto
7. Mapa dos arquivos e do código
8. Como o código trabalha (um exemplo completo)
9. Conectores: passo a passo
10. Ordem recomendada para configurar tudo
11. Como conferir que deu certo
12. Tudo que fica em aberto
13. Glossário
14. Anexos

1. Como usar este guia

O guia tem quatro partes: **o que existe** (seções 2 a 8), **como conectar os serviços** (seções 9 a 11), **o que falta** (seção 12) e **apoio** (glossário e anexos). Se você está perdido nos conectores, vá direto à seção 9 e siga na ordem da seção 10.

Marcas usadas

Marca	Significa
EU FAÇO	Eu (o Claude) já fiz ou posso fazer no código/repositório.
VOCÊ FAZ	Só você consegue fazer: depende de conta, documento, cartão ou painel externo na sua titularidade.
DECISÃO SUA	Decisão sua antes de seguir (muda o que será construído ou o custo).

Sobre os vídeos e links

> **Leia antes de seguir um vídeo**
>
> Os vídeos foram encontrados por busca no YouTube nesta sessão. **Eu não assisti a nenhum deles.** Usei só o título e a descrição que a busca devolveu. Por isso: (1) confira a data, porque as telas dos serviços mudam; (2) quando o vídeo e a tela de hoje forem diferentes, vale a tela de hoje; (3) alguns vídeos são em inglês ou espanhol porque não achei equivalente em português.
>
> Os endereços de documentação oficial (como `developers.cloudflare.com`) são conhecidos, mas **não foram abertos nesta sessão**. Onde o repositório já traz o passo a passo (arquivos `lancamento.md` e `operacao.md`), o guia segue esse texto, que foi conferido contra o código.

O que este guia não faz

- Não substitui o advogado (Termos e Privacidade) nem o contador (nota fiscal).
- Não traz preços finais dos serviços: eles mudam e dependem de câmbio. Onde há número, ele vem de uma fonte citada.
- Não afirma que o sistema está pronto para lançar: a seção 12 lista o que ainda precisa ser feito e verificado.

2. O projeto em uma página

Controle de Produção é um site (não um aplicativo de celular) para pequenas e médias indústrias de montagem acompanharem a produção por etapas: quanto cada etapa avançou, quantas unidades já saíram, quanto tempo falta, quando o lote termina (em dias úteis), com histórico de quem registrou o quê. Várias empresas usam o mesmo sistema, sem ver os dados umas das outras.

Item	Valor
Tipo de produto	SaaS (assinatura mensal) para o navegador, em português do Brasil
Público	Pequenas e médias indústrias de montagem (ex.: robôs, painéis elétricos, transformadores)
Núcleo do produto	Acompanhar etapas. Todo o resto é opcional e vem desligado por padrão
Tecnologia	Python 3.12 · Flask 3 (páginas renderizadas no servidor) · HTMX (partes que atualizam sem recarregar) · SQLAlchemy 2 · Alembic (migrações) · PostgreSQL 16 · ECharts (gráficos)
Cobrança	Asaas (assinaturas)
Hospedagem prevista	Render (site, worker, crons, banco e Redis) + Cloudflare (DNS e R2) + Resend (e-mail) + Sentry (erros)
Tamanho do código	25.079 linhas de Python em `app/`; 47 arquivos de teste; cerca de 1.100 testes automáticos; 23 migrações do banco; 35 documentos de planejamento
Situação dos testes (SQLite)	1.114 aprovados, 84 pulados, 3 falhas esperadas (xfail), nenhuma falha — na branch principal
Situação dos testes (PostgreSQL)	**Não reproduzida na branch principal.** Um agente rodou 1.172 testes em PostgreSQL 16 local numa versão anterior ao último ajuste; eu não repeti. Ver seção 12
No ar?	**Não.** O Render está conectado, mas sem nenhum serviço ou banco criado. Não há site público ainda

3. O que o sistema faz

3.1 Núcleo (sempre ligado)

- **Projetos (lotes):** cada projeto tem quantidade de unidades, prazo e uma lista de etapas.
- **Etapas e atividades:** cada etapa tem peso (tempo ideal) e, opcionalmente, subatividades.
- **Preencher etapas:** a tela onde a equipe registra o avanço. Dois modos: simples (um percentual ou número de peças por etapa) e detalhado (uma atividade por linha).
- **Painel do projeto:** indicadores (progresso geral, unidades concluídas, tempo restante, dias úteis até o prazo) e gráficos.
- **Histórico e desfazer:** cada registro guarda quem fez e quando; é possível desfazer e **restaurar o projeto para uma data**.
- **Importar e exportar planilhas** (CSV e XLSX): importação com mapeamento de colunas, escolha de minutos ou horas e prévia antes de gravar. A planilha é só entrada e saída; nunca é a fonte dos dados.
- **Modelos de projeto:** receitas reutilizáveis (etapas e tempos), versionadas.
- **Equipe:** convites por e-mail e três papéis (proprietário, supervisor, operador).
- **Assinatura:** planos, teste grátis (planos 1 e 2), faturas, troca de plano, modo leitura quando a cobrança falha.
- **Privacidade (LGPD):** exportar os dados da empresa e solicitar exclusão.
- **Segurança de conta:** senha forte, MFA (código no celular), bloqueio após tentativas erradas.

3.2 Módulos opcionais (desligados por padrão; o proprietário liga em Configurações › Módulos)

Módulo	O que acrescenta
Atribuição	Atribuir atividades a pessoas, ver progresso por pessoa e avisar por e-mail
Alertas	Prazo em risco, etapa parada, atraso previsto; relatório semanal por e-mail
Aguardando peça	Marcar uma etapa como parada à espera de peça, com data prevista e lembrete
Fotos e comentários	Evidência de montagem e não conformidades por atividade
Checklist	Checklist de inspeção por etapa, com assinatura (usuário + data e hora)
Unidades	Número de série por unidade e histórico completo de cada uma
API	API pública de leitura (hoje desligada por padrão e só com parte das rotas prontas)

3.3 Planos de assinatura (aprovados pelo dono; ADR-0019)

Plano	Preço/mês	Projetos	Pessoas	Teste	Extras
Essencial	R$ 249,90	5	5	14 dias	—

Plano	Preço/mês	Projetos	Pessoas	Teste	Extras
Profissional	R$ 497,90	10	25	14 dias	—
Empresa	R$ 997,90	ilimitado	ilimitado	não	MFA obrigatório, API, identidade visual adaptada do site (briefing e ligação)
Implantação	sob orçamento (online ou presencial)	ilimitado	ilimitado	não	Tudo do Empresa + implantação pela equipe técnica, 1 mês de acompanhamento e manutenção, treinamento (serviços, sem checagem no código)

O teste grátis de 14 dias nos planos 1 e 2 é um padrão que mantive; você não confirmou (item em aberto).

3.4 Painel interno (só da sua equipe)

- Endereço `/interno`: só abre para contas marcadas como equipe, com MFA. Qualquer outra pessoa recebe "página não encontrada".
- Acompanha as requisições de identidade visual (status, contato) e **cadastra os horários disponíveis para ligação**.
- Não lê o conteúdo do briefing do cliente (decisão do ADR-0019, aguardando a definição do acesso do profissional).

3.5 Identidade visual do cliente (planos Empresa e Implantação)

- **Briefing:** o proprietário preenche nome da empresa e dos produtos, cores, fontes, sites de referência, tom de voz (formal, neutro, descontraído ou em branco), contato e envia logo e fotos de produto.
- **Ligação:** o cliente escolhe um dia e horário entre os que você cadastrou. O servidor impede que dois clientes peguem o mesmo horário. Remarcar: o cliente pede, você decide.

4. Como as peças se encaixam

Pense no sistema como uma pequena fábrica de serviços. Cada caixa abaixo é um lugar diferente; a coluna da direita diz por que ele existe.

Navegador do cliente Abre o site, faz login, registra o avanço.	**Site (Render, "web")** O programa Flask. Responde às páginas e valida todas as regras.	**Banco (PostgreSQL)** Guarda tudo. Separa as empresas com RLS.
Redis (Key Value) Memória rápida: limite de tentativas, fila.	**Worker** Processo de fundo: alertas, relatório semanal, e-mails em fila.	**Crons** Tarefas por horário: exclusões LGPD, backup, monitor.
Cloudflare R2 Guarda arquivos: fotos e cópias de segurança.	**Resend** Envia os e-mails do sistema.	**Asaas** Cobra as assinaturas e avisa o site quando alguém paga.
Sentry Avisa quando o sistema dá erro (sem dados pessoais).	**Monitor de uptime** Visita o site de tempos em tempos e te avisa se cair.	**DNS (Cloudflare) + domínio (Registro.br)** O endereço do site e dos e-mails.

Isolamento entre empresas (a regra mais importante)

Cada linha com dado de cliente carrega o `company_id`. A empresa ativa vem **só da sessão do usuário** (nunca de formulário ou endereço). Se alguém pedir um recurso de outra empresa, a resposta é "não encontrado" (404), para nem revelar que ele existe. Além do código, o banco aplica uma segunda trava, o **RLS** (Row Level Security): mesmo que um erro de programação esquecesse o filtro, o banco não devolveria a linha de outra empresa. O RLS só protege se o usuário do banco usado pelo site **não** for administrador (ver seção 9.4).

Onde mora cada decisão

- Decisões de arquitetura: `docs/plano/adr/` (ADR-0001 a ADR-0019).
- Segurança: `docs/plano/seguranca.md`. Privacidade: `lgpd.md`. Operação: `operacao.md`.
- Lançamento (checklist): `docs/plano/lancamento.md`. Andamento por fase: `progresso.md`.

5. O que foi feito, em ordem

Bloco	O que entregou
Fases 0–2	Esqueleto do projeto; `calc.py` com as fórmulas da planilha original (com teste de paridade contra `acompanhamento_montagem_v4.xlsx`); identidade visual "Prumo" e base de CSS por tokens.
Fase 3	Banco completo com todas as tabelas, planos iniciais e RLS ligável.
Fase 4	Regras de negócio: projetos, etapas, registro/desfazer, importação e exportação, limites de plano.
Fase 5	Contas: cadastro, confirmação por e-mail, convites, MFA, bloqueio por tentativas.
Fases 6–7	Telas completas do site; segurança (CSP, cabeçalhos, Redis) e LGPD (exportar, excluir, retenção, suporte auditado).
Fases 8–9	Assinaturas com Asaas (checkout, webhook, reconciliação); API pública reservada atrás de uma chave de liberação.
Etapas B–E	Planos novos, painel interno, colaboração (autoria, atribuição, arrastar etapas), dashboard completo.
Etapas F–H	Worker, alertas e "aguardando peça"; fotos, checklist e número de série; Dockerfile, `render.yaml`, backup criptografado e verificações de configuração.
Reestruturação R0–R10	Foco em acompanhar etapas (ADR-0014): novo Painel e "Preencher etapas"; importação de qualquer planilha com mapeamento; restaurar projeto para uma data; módulos opcionais desligados por padrão; limpeza do front antigo; página pública na identidade oliva.
Identidade visual	Do "Prumo Noturno" para o verde-oliva com off-white e preto, fontes Newsreader e DM Sans, menos animação (ADR-0016 e ADR-0018).
Rodada mais recente (ADR-0019)	Planos e preços novos; briefing de identidade visual; agendamento de ligação (horários cadastrados por você); importação com ajuda por campo e seletor minutos/horas; pendências finais (cabeçalho em 768 px, "Sob orçamento", horários em formato brasileiro, variáveis novas na configuração do Render).

Como foi verificado

- Suíte de testes automática: 1.114 aprovados em SQLite na branch principal.
- Revisão de segurança por leitura do código novo desta rodada: nenhum achado de alta confiança. Um achado de nível médio (autorização do pedido de remarcação) foi apontado por uma revisão automática, corrigido e coberto por teste.
- Telas conferidas por captura (375, 768 e 1280 px; claro e escuro) para as áreas tocadas.

O que não foi verificado

- Nenhum teste de RLS na branch principal contra PostgreSQL (não havia servidor acessível; ver seção 12).
- Nenhum envio real de e-mail, nenhuma cobrança real, nenhum upload ao R2 real, nenhum evento real ao Sentry.
- Nenhum deploy: o Render está vazio.

6. As regras do projeto

Estas regras estão em `CLAUDE.md` e valem para qualquer pessoa ou agente que mexa no código. Elas existem porque cada uma evita um tipo de problema caro: vazamento entre clientes, número calculado diferente em duas telas, perda de dados ao mudar o banco, segredo exposto.

#	Regra	O que significa na prática
1	**Multiempresa**	Toda tabela com dado de cliente tem `company_id`; a empresa vem da sessão; recurso de outra empresa = 404; toda rota nova ganha teste de acesso cruzado.
2	**Cálculo num só lugar**	Toda fórmula (progresso, tempo restante, dias úteis) mora em `app/services/calc.py`, sem Flask e sem banco, para ser testável.
3	**Banco só por migração**	Mudança de estrutura só por migração revisada (Alembic). Nunca criar tabelas "na mão" em produção.
4	**Segredos**	Só em variáveis de ambiente. Nunca em código, log ou commit.
5	**Sem planilha como fonte**	Excel/CSV servem só para importar e exportar.
6	**Visual por tokens**	Cores, fontes e espaçamentos só pelo arquivo `tokens.css`; o verde-oliva nunca colore dado (barras, rosca, situação).
7	**Limites no servidor**	Limites do plano são checados no backend; esconder um botão nunca substitui a recusa da rota.
8	**Tudo servido pelo próprio site**	Sem CDN em produção, para manter a política de segurança (CSP) estrita.
9	**Backend é a fonte de regras**	O navegador só exibe o que o servidor calculou; nenhum cálculo, limite ou permissão é decidido no navegador (ADR-0009).

Definição de "pronto" (toda fase)

- Testes novos e antigos verdes; teste de isolamento para cada rota nova.
- `/security-review` sem achado aberto.
- Telas conferidas em 375, 768 e 1280 px, tema claro e escuro.
- README e `docs/plano/progresso.md` atualizados; um commit por fase.
- Decisão fora do plano: registrar um ADR e pedir aprovação (foi o caso do ADR-0019).

7. Mapa dos arquivos e do código

As descrições abaixo vêm do texto de abertura de cada arquivo (o "docstring" que quem programou deixou no próprio código). Quando um arquivo não tem esse texto, aparece "sem descrição no arquivo". O número é a quantidade de linhas.

Estrutura geral do repositório

Pasta / arquivo	Para que serve
`app/`	O sistema: rotas, regras, modelos, telas e estilos.
`app/blueprints/`	As rotas (endereços) agrupadas por assunto.
`app/services/`	As regras de negócio. As rotas chamam estes arquivos; eles não conhecem o navegador.
`app/models/`	As tabelas do banco, descritas em Python.
`app/templates/`	As páginas HTML (Jinja): públicas, área logada, e-mails, painel interno.
`app/static/`	CSS por tokens, JavaScript, fontes (DM Sans, Newsreader), ECharts e imagens. Tudo próprio, sem CDN.
`migrations/versions/`	As 23 migrações: o histórico de mudanças do banco.
`tests/`	47 arquivos de teste automático.
`scripts/`	Ferramentas: capturar telas, gerar `render.yaml`, verificar configuração, backup/restauração, smoke test.
`docs/plano/`	Planejamento: PRD, arquitetura, segurança, LGPD, operação, lançamento, ADRs, progresso.
`docs/design/`	Identidade visual, tokens, capturas, contraste.
`Dockerfile`, `render.yaml`, `render.staging.yaml`	Como o sistema é empacotado e publicado no Render (o YAML é gerado por script e conferido pelo CI).
`run.py`, `gunicorn.conf.py`	Ponto de entrada local e configuração do servidor de produção.
`.github/workflows/ci.yml`	Verificação automática no GitHub: auditoria de dependências, testes (SQLite e PostgreSQL 16), migrações e build do Docker.
`CLAUDE.md`	Regras e contexto do projeto para quem programa.

Núcleo do aplicativo (`app/`)

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`__init__.py`	237	Fábrica da aplicação Flask.

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`assets.py`	56	URLs dos assets de produção (CSS/JS minificados com hash no nome). ``python scripts/build_assets.py`` grava ``static/dist/manifest.json``. Com o manifesto, ``asset_urls("site.css")`` devolve ``/static/dist/site.<hash>.css`` (cache de 1 ano, imutável); sem ele (checkout limpo) devolve os…
`cli.py`	296	Comandos de linha (flask …).
`config.py`	359	Configuração por ambiente. Tudo vem de variáveis de ambiente (ver .env.example).
`configcheck.py`	390	Etapa H — conferência da configuração de produção (``flask config-check``). Lista TUDO o que falta ou está inseguro, de uma vez, em vez de falhar no primeiro erro como o ``finalize_config`` (que continua sendo a trava de boot). Usado no ``preDeployCommand`` do ``render.yaml``: se houver erro, o…
`extensions.py`	24	Instâncias das extensões Flask (inicializadas em create_app).
`formatting.py`	89	Formatação pt-BR para os templates (sem dependência de locale do sistema).
`forms.py`	192	Formulários (Flask-WTF: CSRF em todo POST).
`forms_accounts.py`	119	Formulários da Fase 5 (contas, equipe, MFA). Separados de forms.py para não colidir com a Fase 4.
`forms_company.py`	40	Formulários das configurações da empresa (/app/empresa).
`forms_production.py`	221	Formulários da Fase 4 (modelos, edição, importação). Separados de ``forms.py`` para não misturar com contas/login. Todos herdam o CSRF do Flask-WTF.
`logging_config.py`	239	Fase 7 — logs estruturados (JSON) com ``request_id`` e Sentry opcional com filtro. Regras (docs/plano/seguranca.md, docs/plano/lgpd.md): - Uma linha JSON por evento em stdout (o host guarda; retenção de 6 meses). - ``request_id``: o ``X-Request-ID`` do proxy só é aceito se tiver formato seguro…
`ops_cli.py`	127	Etapa H — comandos de operação: ``config-check``, ``mail-test``, ``storage-test``. Nenhum deles imprime segredo: só nomes de variáveis, o domínio do remetente e o resultado.
`rls.py`	182	Row Level Security (RLS) do PostgreSQL: a segunda barreira do isolamento (ADR-0003). Como funciona - A migration ``b3e9f1a2c4d5`` cria as políticas em toda tabela com dado de cliente: ``company_id = NULLIF(current_setting('app.company_id', true), '')::uuid`` (USING e WITH CHECK). Extras…
`security.py`	159	Fase 7 — camada HTTP de segurança num lugar só. - CSP estrita com nonce por requisição (sem ``unsafe-inline``); ``csp_nonce()`` no Jinja para o raro ``<script nonce="…">`` (hoje nenhum template precisa: todo JS é arquivo). - HSTS fora de development/test, Permissions-Policy…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`tenancy.py`	167	Isolamento multiempresa num único lugar. - A empresa ativa vem SÓ de ``session["company_id"]`` (nunca de formulário, URL ou header). - ``load_company`` roda antes de cada requisição e confere se o usuário tem vínculo ativo com essa empresa; senão a empresa é descartada. - Consultas a dado de…

Rotas (`app/blueprints/`)

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`__init__.py`	0	sem descrição no arquivo
`account.py`	388	Conta do usuário (/conta): confirmar e-mail, perfil, sessões, MFA e troca de empresa. Também registra as guardas que rodam DEPOIS de ``tenancy.load_company``: - e-mail não confirmado há mais de 7 dias → só a tela de confirmação; - empresa com ``mfa_required`` e usuário owner sem MFA → só a tela de…
`api_v1.py`	500	API pública v1 (Fase 9) e tela de tokens. Registrada SÓ com ``FEATURE_PUBLIC_API`` ligada. API (``/api/v1``, contrato em docs/api/openapi.yaml): - Autenticação exclusiva por ``Authorization: Bearer cp_...``. Cookie de sessão NÃO autentica (a empresa vem do token, nunca da sessão) e o blueprint é…
`app.py`	776	Área logada (/app). Toda rota passa por tenant_required e consulta via tenancy.
`auth.py`	260	Cadastro de empresa, entrar (com bloqueio progressivo e MFA), sair, esqueci a senha.
`billing.py`	273	Telas de assinatura (Fase 8): assinatura atual, trocar plano (com projeção), dados de cobrança, faturas e cancelar. Só o proprietário (owner) gerencia a cobrança: supervisor e operador recebem 403. Todas as rotas aceitam empresa em modo leitura (é por aqui que ela regulariza). A empresa vem só da…
`dados.py`	445	Importar e exportar (Fase 4) — supervisor+. Exportar: CSV e XLSX do projeto (mesmas colunas da tela, células perigosas neutralizadas). Importar: upload limitado e lido em memória → pré-visualização com erros por linha → confirmação grava tudo numa transação. Entre a pré-visualização e a confirmação…
`dev.py`	67	Páginas só para desenvolvimento. /estilo mostra tokens e componentes (design-system.md). A rota existe em todos os ambientes, mas responde 404 fora de development (mesma resposta de uma página inexistente), para não expor nada em produção.
`edicao.py`	547	Edição de projeto (Fase 4): dados, renomear/reordenar/arquivar etapas e atividades, duplicar, arquivar, comparar/aplicar versão do modelo, desfazer registro (D13) e restaurar o projeto para uma data (R8). Tudo supervisor+, POST com CSRF, recursos resolvidos por ``get_or_404`` (empresa da sessão…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`empresa.py`	171	Configurações da empresa (/app/empresa) — só o proprietário (telas.md §13). Dados (nome, CNPJ), calendário (jornada, dias de trabalho) e feriados da empresa. A empresa é SEMPRE a da sessão; o id do feriado é resolvido nela (de outra empresa, ou nacional → 404). R6 (ADR-0017): Configurações ›…
`identidade.py`	422	ADR-0019 — briefing de identidade visual e agendamento da ligação (/app/identidade-visual). Acesso (checado aqui, no servidor; o menu só esconde por conveniência — ADR-0009): 1. Login e empresa da sessão (``tenant_required``). 2. Plano em vigor sem a feature ``visual_custom`` → 404 para TODOS os…
`interno.py`	259	Etapa C — painel interno da equipe (/interno). Só para ``User.is_staff`` com MFA (``staff.is_staff``): qualquer outra pessoa, logada ou não, recebe 404 (a rota não revela que existe). Painel só para ACOMPANHAR as requisições de identidade visual: contato do cliente, estado e anotação. Não entra em…
`lgpd.py`	204	Fase 7 — Privacidade e dados da empresa (/app/privacidade), só proprietário. - Exportar: POST (CSRF + auditoria) devolve ZIP de CSVs só com dados da empresa da sessão. - Excluir: pede senha + nome da empresa; agenda a exclusão com carência de 30 dias (cancelável). O apagamento definitivo é do…
`modelos.py`	210	Modelos de etapas (/app/modelos) — supervisor+ (telas.md §11).
`preencher.py`	607	Telas da base (reestruturação R3–R5): Preencher etapas, Meta e equipe e Como funciona. Preencher etapas (plano §4.3): lista das etapas à esquerda e a tabela da etapa à direita, com células amarelas que salvam AO SAIR DO CAMPO (static/js/preencher.js manda um POST pequeno por célula via HTMX; a…
`public.py`	194	Site público, /saude (liveness), /health (prontidão) e troca de tema.
`qualidade.py`	502	Etapa G — qualidade: evidências por atividade (comentários e fotos). R6 (ADR-0017): três módulos OPCIONAIS moram aqui — ``fotos`` (evidências), ``checklist`` (checklist e inspeções) e ``unidades`` (número de série). Com o módulo desligado na empresa, a rota responde 404 (``tenancy.tenant_required``…
`team.py`	199	Usuários e convites (/app/usuarios) e aceite de convite (/convite/<token>). Papéis: supervisor vê a lista, convida supervisor/operador e reenvia/revoga esses convites; mudar papel, desativar/reativar membro e exigir MFA são só do proprietário. Tudo pela empresa da sessão (``get_or_404``): membro ou…
`webhooks.py`	89	Webhooks recebidos: ``POST /webhooks/asaas``. 1. Sem CSRF (quem chama é o Asaas), mas com rate limit e corpo pequeno. 2. Header ``asaas-access-token`` comparado em tempo constante (``hmac.compare_digest``) com ``ASAAS_WEBHOOK_TOKEN``; sem token configurado ou diferente → 401. 3.…

Regras de negócio (`app/services/`)

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`__init__.py`	0	sem descrição no arquivo
`accounts.py`	676	Regras de conta e equipe (Fase 5): cadastro, confirmação de e-mail, reset de senha, convites, membros, perfil e sessões. Tokens: ver ``app/services/tokens.py`` (só o SHA-256 no banco, uso único, expiração). Mensagens de erro em português vão direto para a tela (``AccountError``). RLS (ADR-0003)…
`agenda.py`	404	ADR-0019 §3 — agendamento da ligação de identidade visual. - Horários (``call_slots``) são dado da PLATAFORMA: só a equipe cria e remove (``/interno``, MFA, ``platform_audit_logs``). Guardados em UTC; a equipe digita e a tela mostra em America/Sao_Paulo. - Reserva (``call_bookings``) é dado DA…
`alerts.py`	314	Alertas automáticos por empresa (Etapa F). Módulo opcional ``alertas`` (R6, ADR-0017): empresa com o módulo desligado é pulada. A REGRA mora em ``calc.alertas_projeto`` (pura). Aqui só: montar as entradas a partir do banco (no contexto RLS da empresa), gravar o que saiu com dedupe ``(company_id…
`api_tokens.py`	172	Tokens da API pública (Fase 9): criação, autenticação e revogação. - O token é ``cp_`` + 43 caracteres aleatórios (``secrets.token_urlsafe(32)``). O texto puro só existe na resposta da criação (mostrado uma vez); no banco fica o SHA-256 em hex (``token_hash``) e um prefixo curto (``token_prefix``)…
`audit.py`	206	Trilha de auditoria (``audit_logs``, append-only). Ponto único para registrar ações administrativas: mudança de papel, convite, plano, exclusão, aplicar versão de modelo, desfazer registro, arquivar etc. A Fase 5 (usuários, convites) chama ``record`` do mesmo jeito: from app.services import audit…
`calc.py`	999	Regras de cálculo do Controle de Produção. Reproduz as fórmulas da planilha de referência ``docs/referencia/acompanhamento_montagem_v4.xlsx`` (ver ``docs/plano/arquitetura.md`` §1.1). Este módulo é PURO: não importa Flask, SQLAlchemy nem nada de banco. Recebe tipos simples (números, datas…
`charts.py`	36	Geometria do medidor SVG do painel (Etapa E; R9 manteve só o medidor). PURO como calc.py: sem Flask, sem banco, só números e dicts. Aqui só há **desenho** (o arco do medidor); toda regra de negócio (progresso, horas, dias úteis, status) já vem pronta de ``app/services/calc.py``. O template apenas…
`checklists.py`	355	Etapa G — checklist de inspeção por etapa. Papéis (decisão registrada no ADR-0011; conferidos na ROTA e aqui só a regra de negócio): - supervisor+ define itens da etapa do projeto (incluir/arquivar) e liga “exige checklist”; - operador+ abre uma inspeção (rascunho), preenche e ASSINA; - supervisor+…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`comments.py`	84	Etapa G — comentários por atividade (evidência de montagem / não conformidade). - Operador+ comenta (só membros da empresa da sessão; a rota resolve a atividade com ``tenancy.get_or_404``). Texto limpo de espaços nas pontas, 1..``COMMENT_MAX_CHARS``. - Comentário não se edita. Supervisor+ apaga: a…
`company.py`	169	Configurações da empresa (/app/empresa): dados, jornada, dias de trabalho e feriados. Só o proprietário muda (a rota exige). Toda alteração entra em ``audit_logs`` na mesma transação. Feriados nacionais são globais (``company_id`` NULL) e não aparecem aqui como editáveis: a empresa só cria e remove…
`dashboard.py`	356	Monta os dados do Painel (reestruturação R2) a partir dos resultados de ``calc``. Sem Flask e sem banco: recebe objetos simples (etapas com ``id``/``name``, o ``ResultadoProjeto`` já calculado por ``app/services/calc.py``, datas e funções que montam links) e devolve dicts prontos para o template e…
`email.py`	271	Envio de e-mail transacional. Interface ``EmailSender`` com dois backends: - ``ConsoleEmailSender`` (development/test): guarda as mensagens em memória (``outbox(app)``, usado pelos testes) e, em development, grava um ``.eml`` em ``instance/emails/`` para abrir o link no navegador. NÃO imprime o…
`entitlements.py`	231	Limites de plano e modo leitura (regra inviolável 7: checado no backend). Plano efetivo da empresa: - com assinatura (``subscriptions``): o plano dela; - sem assinatura nenhuma: teste grátis implícito de ``TRIAL_DAYS`` dias contados da criação da empresa, no plano de teste (``trial_plan``). Teste…
`events.py`	51	Eventos de domínio (outbox). ``emit`` só ADICIONA a linha em ``domain_events`` na sessão atual: ela é gravada no mesmo commit da mudança (ou some junto num rollback). Nada consome os eventos no MVP; o worker da Fase 9 lê ``published_at IS NULL`` e entrega aos webhooks de saída. O payload leva só…
`export_layout.py`	491	Exportação no LAYOUT DA BASE (R7, plano §4.6): o .xlsx sai igual à planilha de acompanhamento de referência (``docs/referencia/acompanhamento_montagem_v4.xlsx``), COM AS FÓRMULAS, para continuar funcionando no Excel/LibreOffice depois de baixado. Aba “Acompanhamento”: título (nome do projeto)…
`exporter.py`	262	Exportação de projeto em CSV e XLSX (mesmas colunas da tela + o que a importação lê). Uma linha por atividade (etapa sem atividade sai numa linha só com a etapa). Inclui o que foi ARQUIVADO (coluna “Situação”), para o histórico não se perder e para que exportar → importar devolva o mesmo projeto.…
`holidays.py`	81	Feriados nacionais (linhas globais com company_id NULL) e feriados da empresa. Nacionais: Lei 662/1949 (com 6.802/1980 e 14.759/2023) — 1º/1, 21/4, 1º/5, 7/9, 12/10, 2/11, 15/11, 20/11 e 25/12 — mais a Sexta-feira da Paixão (móvel, guardada em todo o país). Carnaval e Corpus Christi são ponto…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`import_mapping.py`	631	Importar QUALQUER planilha (.xlsx/.csv) com mapeamento de colunas (R7, plano §4.6). Etapa 1 — ``inspect``: com a grade já lida (``importer.load_grid``, mesmas proteções de tamanho, zip bomb, DTD e senha), devolve as abas e, para a aba escolhida, a linha de cabeçalho (detectada ou escolhida), as…
`importer.py`	828	Importação de projeto: CSV (modelo do site / exportação), XLSX exportado pelo site (lista plana ou layout da base), a planilha de referência ``acompanhamento_montagem_v4.xlsx`` e — R7 — QUALQUER planilha .xlsx/.csv com mapeamento de colunas (``app/services/import_mapping.py``). Fluxo: ``load_grid``…
`lgpd.py`	514	Fase 7 — LGPD: portabilidade (ZIP de CSVs), exclusão com carência, expurgo e retenção. Regras (docs/plano/lgpd.md): - Exportar: só o proprietário, só linhas com ``company_id`` da empresa da sessão + a própria empresa + os usuários vinculados a ela. Segredos (hash de senha/token, segredo TOTP) nunca…
`lockout.py`	117	Bloqueio progressivo de login por conta e por IP (tabela ``login_throttles``). - Cada senha (ou código MFA) errada soma uma falha na chave da conta (e-mail normalizado, exista ou não a conta — o comportamento não revela cadastro) e na chave do IP. - Ao atingir o limite (5 por conta, 20 por IP —…
`mfa.py`	192	MFA TOTP (pyotp) com segredo cifrado e códigos de recuperação. - Segredo TOTP cifrado com Fernet; a chave é derivada por HKDF-SHA256 de ``MFA_ENCRYPTION_KEY`` (ou, sem ela, de ``SECRET_KEY``). Fase 7: MultiFernet — a lista "nova,antiga" decifra com as duas e cifra com a nova; ``flask…
`modules.py`	251	R6 — módulos opcionais por empresa (ADR-0017, plano de reestruturação §4.5). O núcleo do produto é acompanhar ETAPAS. Todo o resto é opcional, DESLIGADO por padrão e ligado pelo proprietário em Configurações › Módulos. Este arquivo é o ponto único da regra: - ``CATALOG``: cada módulo, o nome em…
`notifications.py`	361	Notificações por e-mail disparadas pelo worker (Etapa F). Rodam FORA de requisição, no contexto RLS da empresa do evento (ADR-0010). Toda mensagem leva só o necessário (nome da empresa/projeto/atividade e o link); links sempre absolutos a partir de ``APP_BASE_URL`` (``email.external_url``). Falha…
`passwords.py`	28	Hash de senha com Argon2id (argon2-cffi).
`photos.py`	281	Etapa G — fotos de evidência por atividade. Validação (tudo no servidor; o ``accept`` do navegador é só ajuda): 1. Tamanho ≤ ``PHOTO_MAX_BYTES`` (5 MB) — além do teto do corpo da requisição (413). 2. Tipo pelo CONTEÚDO (magic bytes), nunca pela extensão nem pelo Content-Type enviados: só JPEG, PNG…
`plans.py`	137	Catálogo de planos (fonte única para `flask seed-plans`, para a migration da Etapa B e para os testes). Quatro planos (ADR-0019, que substitui D2–D4 do PRD e a Etapa B): - ``essencial`` — R$ 249,90/mês, 5 projetos ativos, 5 pessoas, teste grátis de 14 dias; - ``profissional`` — R$ 497,90/mês, 10…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`projects.py`	1350	Casos de uso de projetos. Toda consulta passa por tenancy (company_id da sessão).
`restore.py`	471	R8 — restaurar o projeto para como estava numa data e hora. Escopo: só as PEÇAS concluídas das atividades (o que o histórico ``progress_entries`` registra). Nomes, tempos ideais, quantidade, estrutura de etapas/atividades (criar, mover, arquivar) e marcas como “aguardando peça” NÃO voltam: o…
`staff.py`	166	Etapa C — equipe da plataforma e requisições de identidade visual. - ``grant``/``revoke``/``list_staff``: só por linha de comando (``flask staff …``), com motivo e quem autorizou; a conta precisa de e-mail confirmado e MFA ligado. Cada ação vai para ``platform_audit_logs`` (append-only).…
`storage.py`	352	Etapa G — armazenamento de arquivos (fotos de evidência) atrás de uma interface. Backends: - ``local``: disco, em ``STORAGE_LOCAL_DIR`` (padrão ``instance/uploads`` — FORA de ``static``, nunca servido direto). Só development/test: ``config.finalize_config`` recusa em produção. - ``s3``: S3…
`support.py`	209	Fase 7 — acesso temporário de suporte da plataforma a uma empresa. Só por linha de comando (``flask suporte conceder|revogar|listar|expirar``), nunca pela tela: - motivo obrigatório (mín. 15 caracteres), prazo em horas (teto ``SUPPORT_ACCESS_MAX_HOURS``), papel operador ou supervisor (nunca…
`templates.py`	458	Modelos de etapas versionados. - ``create_template`` cria o modelo com a versão 1 e a marca como atual. - ``update_template`` grava uma NOVA versão quando as etapas mudam (versões são imutáveis; projetos criados de versões antigas não mudam). - ``stages_from_version`` devolve a CÓPIA das…
`tokens.py`	47	Tokens de uso único (confirmação de e-mail, reset de senha, convite). - 32 bytes aleatórios (``secrets.token_urlsafe``) vão SÓ no link do e-mail. - No banco fica só o SHA-256 em hex (64 caracteres). O token tem entropia alta, então um hash rápido basta (não é senha); a busca é pela igualdade do…
`trusted_device.py`	76	Cookie de dispositivo confiável (para o bloqueio de login por IP). Com o IP bloqueado (muitas senhas erradas vindas dele), o login NÃO verifica credenciais — resposta genérica de bloqueio, o que barra password spraying —, EXCETO para contas que já entraram com sucesso neste navegador. Assim, um…
`units.py`	238	Etapa G — unidades (número de série por equipamento do projeto). - Rastreabilidade, NÃO cálculo: progresso, status e prazo continuam vindo de ``activities.units_done`` via ``calc.py`` (intocado). A unidade só identifica QUAL equipamento recebeu um lançamento ou uma inspeção. - Supervisor+ cria…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`visual.py`	436	ADR-0019 §2 — briefing de identidade visual (dado da empresa). Regras (todas aqui, no servidor; a tela só mostra o que este módulo decide — ADR-0009): - Só empresa com a feature ``visual_custom`` no plano em vigor (``available``); a rota responde 404 sem ela. Só o proprietário preenche (papel…
`worker.py`	352	Worker (Etapa F, ADR-0010): outbox + tarefas agendadas, empresa a empresa no contexto RLS. Roda em PROCESSO SEPARADO (``flask worker run``; ``--once`` faz um ciclo e sai). Nunca usa papel com BYPASSRLS: lê ``companies`` (sem RLS) e, para cada empresa, faz ``rls.set_context(company_id=…)``, trabalha…

Cobrança (`app/services/billing/`)

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`__init__.py`	10	Assinaturas e pagamento (Fase 8, ADR-0004). - ``provider``: interface ``PaymentProvider`` e tipos. - ``asaas`` / ``fake``: implementações (Asaas real, sandbox ou produção; fake em memória). - ``service``: regras (teste grátis, assinar, trocar plano, cancelar, webhook, carência, reconciliação, ações…
`asaas.py`	231	``AsaasProvider``: API v3 do Asaas (https://docs.asaas.com, ver docs/plano/pagamentos.md). - Base URL por ``ASAAS_BASE_URL`` (padrão: sandbox ``https://api-sandbox.asaas.com/v3``; produção ``https://api.asaas.com/v3``). Chave em ``ASAAS_API_KEY``, enviada no header ``access_token`` (o Asaas não usa…
`cli.py`	99	Comandos de cobrança. - ``flask billing-reconcile``: confere toda assinatura com o provedor (status da assinatura e das cobranças) e recalcula o estado local (carência vencida → modo leitura, teste vencido, cancelada no fim do período). Idempotente: rodar de novo sem mudança no provedor não altera…
`documents.py`	60	CPF e CNPJ: normalização e dígitos verificadores (sem Flask nem banco). Usado no formulário (mensagem no campo) e de novo no serviço (``service.BillingInfo``), então um caminho que pule o formulário também é recusado.
`fake.py`	214	``FakeProvider``: provedor em memória, determinístico, para testes e desenvolvimento. Ids previsíveis (``cus_fake_0001``, ``sub_fake_0001``, ``pay_fake_0001``, ``evt_fake_0001``). O estado vive em ``FakeStore`` (um por app, em ``app.extensions``), então sobrevive entre requisições do mesmo app de…
`notify.py`	100	Avisos de cobrança por e-mail (pagamento em atraso / carência). Usa o serviço de e-mail da Fase 5 (``app.services.email.send_template``): console em development/test (os testes leem ``email.outbox(app)``), Resend em produção, entrega sem derrubar a requisição. Textos em…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`provider.py`	98	Interface ``PaymentProvider`` (ADR-0004) e os tipos que ela devolve. O resto do sistema (serviço, telas, webhook, reconciliação) só conhece esta interface: trocar o Asaas por outro provedor é escrever outra implementação. Valores sempre em centavos (int) e datas como ``date``; o status de cobrança…
`service.py`	1082	Regras de assinatura e cobrança (Fase 8). Estados locais (``subscriptions.status``), arquitetura.md §7: trialing → active ⇄ past_due → (carência) → expired; canceled vale até current_period_end - O estado é NOSSO e é sempre RECALCULADO a partir das cobranças locais (``recompute``), não “empurrado”…

Tabelas (`app/models/`)

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`__init__.py`	74	Modelos. Importar daqui garante que todos estejam registrados no metadata.
`accounts.py`	205	Empresas, usuários e vínculos.
`base.py`	29	Tipos e mixins comuns aos modelos.
`billing.py`	135	Planos, assinaturas, cobranças e webhooks recebidos (Asaas via PaymentProvider).
`modules.py`	41	R6 — módulos opcionais por empresa (ADR-0017). Uma linha por empresa e módulo que alguém já ligou ou desligou. Sem linha = DESLIGADO: empresa nova vê só o núcleo (etapas). O catálogo de módulos (chaves, nomes, rotas) mora em ``app/services/modules.py``; aqui só a persistência. Dado da empresa…
`notifications.py`	72	Etapa F — worker: marcas de execução das tarefas agendadas e alertas automáticos. Ambas são dado da empresa (``company_id NOT NULL``, RLS ``tenant_isolation``): o worker grava cada empresa no contexto dela (ADR-0010).
`platform.py`	102	Auditoria, eventos de domínio (outbox) e clientes/tokens da API pública.
`privacy.py`	44	Fase 7 — trilha de auditoria da PLATAFORMA (fora das empresas). ``audit_logs`` pertence à empresa e é apagado com ela no expurgo LGPD. O que precisa sobreviver ao expurgo (prova de que a empresa X foi apagada, quando e quantas linhas) e as ações de operador sem empresa ficam aqui. Sem dado pessoal…
`production.py`	221	Projetos, etapas, atividades, registros de progresso e feriados.
`quality.py`	242	Etapa G — qualidade: evidências por atividade (comentários e fotos). Tudo é dado da empresa (``company_id NOT NULL``, RLS ``tenant_isolation``, índices começando por ``company_id``). Comentário e foto nunca mudam depois de criados; supervisor+ pode apagar (linha removida + ``audit_logs`` sem o…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`security.py`	32	Fase 5 — bloqueio progressivo de login (por conta e por IP), persistido no banco.
`staff.py`	47	Etapa C — dados da PLATAFORMA acompanhados pela equipe (painel ``/interno``). ``visual_requests``: requisição de identidade visual aberta quando a empresa contrata um plano com ``visual_custom`` (Empresa, Implantação). É dado da plataforma, não da empresa: só a equipe (``User.is_staff`` + MFA) lê e…
`templates.py`	115	Modelos de etapas (templates) versionados. Um modelo (``templates``) tem versões imutáveis (``template_versions``); cada versão tem etapas e atividades com tempo ideal. Criar um projeto COPIA as etapas/atividades da versão para ``stages``/``activities`` (snapshot) e guarda…
`visual.py`	163	ADR-0019 §2–3 — briefing de identidade visual e agendamento da ligação. Dado DA EMPRESA (``company_id NOT NULL`` + RLS ``tenant_isolation``, migration c5a1e9d7b3f2): - ``visual_briefings``: um briefing por empresa, preenchido só pelo proprietário. Ligado à requisição da plataforma…

Scripts (`scripts/`)

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`build_assets.py`	87	Gera os assets de produção: CSS/JS minificados com hash do conteúdo no nome. Uso: python scripts/build_assets.py [--check] Saída em ``app/static/dist/``: - ``site.<hash>.css`` = tokens + base + layout + components + pages + publico + utilities, minificado - ``app.<hash>.js`` = ``static/js/app.js``…
`capturar_telas.py`	242	Captura cada tela em 375, 768 e 1280 px, tema claro e escuro (Playwright + Chromium). Uso: python scripts/capturar_telas.py <pasta> (CAP_ONLY=a,b CAP_WIDTHS=375 CAP_SCHEMES=light filtram) Sobe o app com dados fictícios (telas.md §0: lote CX-10, 5 etapas) num SQLite temporário. Os PNGs são reduzidos…
`config_check.py`	37	Conferência da configuração SEM subir o app (útil quando o boot já falha por falta de variável). Uso: python scripts/config_check.py [--online] [--strict] [--ignorar CODIGO …] [--env production] Mesma lógica de ``flask config-check`` (app/configcheck.py); lê as variáveis do ambiente. Sai com 1 se…
`gen_render_yaml.py`	271	Gera render.yaml (produção) e render.staging.yaml (homologação) a partir de UMA descrição. Por que gerar: web, worker e crons repetem dezenas de variáveis; editar três lugares à mão em dois arquivos é como as configurações divergem. Aqui a lista está num só lugar e o CI roda ``python…

Arquivo	Linhas	O que faz (texto do próprio arquivo)
`smoke_check.py`	189	Smoke test de um ambiente no ar (staging ou produção), sem navegador e sem login. Uso: python scripts/smoke_check.py https://staging.seudominio.com.br python scripts/smoke_check.py https://www.seudominio.com.br --producao --outro-host https://seudominio.com.br Confere o que dá para provar de fora…

Migrações do banco (`migrations/versions/`)

Arquivo	Descrição
`228fffa5a2d8_fase_7_seguranca_e_lgpd.py`	Fase 7 — segurança e LGPD: acesso de suporte, auditoria da plataforma e token da API com RLS. - memberships.is_support / memberships.expires_at: acesso temporár
`4d8a644d753b_fase_5_contas.py`	Fase 5 — contas: bloqueio progressivo, último passo TOTP, último acesso e convite por token com RLS. - login_throttles: contador de falhas de login por conta e
`a5d9e3f7c2b8_etapa_f_alertas.py`	Etapa F — alertas automáticos e preferências de e-mail. - ``alerts`` (dado da empresa, RLS ``tenant_isolation``): alerta por projeto/etapa com dedupe ``UNIQUE(c
`a7c2d4e8f901_fase_3_seed_planos.py`	Fase 3 — dados: planos Essencial, Profissional e Empresa. Mesmos valores e ids (uuid5 do código) de app/services/plans.py::PLAN_CATALOG, copiados aqui de propós
`a9d3f5c1b7e2_adr0019_planos_novos.py`	ADR-0019 — planos novos: novos preços e limites de Essencial, Profissional e Empresa. Só dados (sem mudança de esquema). Os códigos e os ids (uuid5 do código) n
`b1e7c9d2a4f6_etapa_b_planos_novos.py`	Etapa B — planos novos: Essencial, Profissional, Empresa e Implantação (sob consulta). Esquema: ``plans.price_cents`` passa a aceitar NULL (= sob consulta, sem
`b3e9f1a2c4d5_fase_3_politicas_rls.py`	Fase 3 — políticas de Row Level Security (segunda barreira do isolamento, ADR-0003). Cria as políticas em toda tabela com dado de cliente, mas NÃO liga a cobran
`b5d2e7a9c3f1_r4_modo_de_progresso.py`	R4 — modo de lançar progresso por projeto (plano de reestruturação §4.4, decisão D1). ``projects.progress_mode``: ``'simples'`` (a pessoa lança o % da etapa) ou
`b6e1f4a2c9d3_etapa_g_evidencias.py`	Etapa G — evidências por atividade: comentários e fotos. - ``activity_comments``: texto (1..1000), autor, data. Apagar = linha removida + auditoria. - ``activit
`c4f8a1d2e3b6_fase_4_desfazer_registro.py`	Fase 4 — desfazer registro de peças (D13) por lançamento compensatório. progress_entries continua append-only: desfazer grava uma linha nova com delta negativo

Arquivo	Descrição
`c5a1e9d7b3f2_adr_0019_briefing_e_agendamento.py`	ADR-0019 — briefing de identidade visual e agendamento da ligação. - ``visual_requests.status`` aceita ``briefing_filled`` (nova → briefing preenchido → em anda
`c7d3e9a1b5f2_etapa_c_painel_interno.py`	Etapa C — equipe da plataforma e requisições de identidade visual. ``users.is_staff`` (padrão false) e ``visual_requests`` (dado da plataforma, sem RLS por empr
`c8f2a5d1e7b4_etapa_g_checklist.py`	Etapa G — checklist de inspeção por etapa. - ``template_checklist_items`` (itens por etapa do MODELO; versionados com a versão) e ``template_stages.checklist_re
`d4a8c2e6f1b3_etapa_d_colaboracao.py`	Etapa D — colaboração: versão da etapa e responsável da atividade. - ``stages.version`` (padrão 1, CHECK >= 1): controle otimista ao renomear a etapa (409 com o
`d8b2e6f4a1c7_fase_8_cobranca.py`	Fase 8 — cobrança: link de pagamento, desconto e busca do webhook com RLS. - payments.invoice_url: página de pagamento do provedor (Pix, boleto ou cartão). - su
`d9a3b6e2f8c5_etapa_g_unidades.py`	Etapa G — unidades (número de série) e vínculo opcional com lançamentos e inspeções. - ``units``: série única por empresa+projeto (UNIQUE), status, versão; índi
`e05ae54663a3_fase_3_schema_completo.py`	Fase 3 — schema completo do MVP: modelos de etapas versionados, convites, tokens de conta, MFA, planos, assinaturas, cobranças, webhooks, auditoria, outbox e AP
`e2b7f1c3a9d0_etapa_f_worker.py`	Etapa F — worker (ADR-0010): backoff do outbox e marcas de execução. - ``domain_events.next_attempt_at`` (nullable): próxima tentativa depois de falha (backoff)
`e8c4a7b2d1f9_r8_restaurar.py`	R8 — restaurar o projeto para uma data e hora. A restauração grava lançamentos COMPENSATÓRIOS em ``progress_entries`` (que continua append-only). Duas colunas o
`e972bfba83c7_esqueleto_inicial.py`	Fase 0 — esqueleto inicial: empresas, usuários, vínculos, projetos, etapas, atividades, registros de progresso (append-only) e feriados. Gerada por autogenerate
`f1a7c3e9b2d4_r6_modulos_opcionais.py`	R6 — módulos opcionais por empresa (ADR-0017, plano de reestruturação §4.5). - ``company_modules``: uma linha por empresa e módulo (``atribuicao``, ``alertas``,
`f3c8a2d4b1e5_etapa_f_aguardando_peca.py`	Etapa F — etapa “aguardando peça” (só sinaliza; nunca bloqueia o registro). - ``stages.waiting_part`` (bool, padrão false) - ``stages.part_expected_on`` (date,
`f6a1b2c3d4e5_fase_6_produto_do_projeto.py`	Fase 6 — campo opcional "produto" no projeto (subtítulo na lista e no detalhe). Só ADD COLUMN anulável: não toca em dados existentes nem em RLS (projects já tem

A cadeia termina em `c5a1e9d7b3f2` (briefing e agendamento), depois de `a9d3f5c1b7e2` (planos novos). Há uma única “cabeça” de migração, requisito para o deploy automático.

Páginas (`app/templates/`) por pasta

Pasta	Arquivos
`/`	3
`/account`	6
`/app`	17
`/app/detail`	7
`/auth`	6
`/components`	20
`/dados`	4
`/dev`	1
`/edicao`	4
`/emails`	21
`/empresa`	2
`/errors`	7
`/identidade`	3
`/interno`	4
`/lgpd`	1
`/modelos`	4
`/preencher`	9
`/public`	7
`/qualidade`	6
`/team`	2

Testes (`tests/`)

Arquivo	Testes	Cobre
`test_accounts.py`	45	Fase 5 — contas: e-mail, confirmação, reset de senha, convites, membros, perfil, sessões, MFA, bloqueio progressivo, troca de empresa e ausê
`test_adr19_briefing_agenda.py`	40	ADR-0019 — briefing de identidade visual e agendamento da ligação. - Briefing (dado da empresa): só plano com `visual_custom` (senão 404),
`test_api_v1.py`	30	API pública v1 (Fase 9): flag, autenticação por token, escopos, plano, isolamento, contrato.

Arquivo	Testes	Cobre
`test_auth.py`	17	Cadastro, login, papéis e CSRF.
`test_billing.py`	36	Fase 8 — assinatura e pagamento com FakeProvider (sem rede). Cenários do roadmap: aprovado, recusado, vencido, pago com atraso, cancelado, r
`test_billing_asaas.py`	10	Fase 8 — AsaasProvider sem rede (sessão HTTP falsa): URL, header access_token, timeout, corpo das chamadas, mapeamento de status e que a cha
`test_billing_cli.py`	6	Fase 8 — `flask billing-reconcile` (idempotente) e `flask billing-admin` (motivo auditado).
`test_billing_rls.py`	2	Fase 8 com RLS ligado e forçado (papel comum, sem BYPASSRLS): o webhook chega sem empresa e acha a assinatura só pelo id do provedor (políti
`test_calc.py`	38	Testes de app/services/calc.py. 1. Casos conferidos à mão (contas no comentário de cada teste). 2. calc.py é puro: não importa flask nem sql
`test_charts.py`	2	Geometria do medidor SVG (app/services/charts.py): pura e coerente.
`test_etapa_b_planos.py`	20	Etapa B — planos novos e cobrança. Essencial (R$ 249,90; 5 projetos; 5 pessoas; teste), Profissional (R$ 497,90; 10; 25; teste), Empresa (R$
`test_etapa_c_interno.py`	7	Etapa C — painel interno da equipe e requisição de identidade visual. - /interno responde 404 para quem não é equipe (anônimo, dono de empre
`test_etapa_d.py`	17	Etapa D — colaboração: autoria, edição com concorrência, atribuição, progresso por pessoa e arrastar (ordem decidida no servidor).
`test_etapa_e.py`	14	Etapa E — painel completo do projeto: KPIs, gráficos SVG no servidor e tabela de etapas. Cálculo só em calc.py (paridade com a planilha em t
`test_etapa_f_alertas.py`	19	Etapa F — alertas automáticos: regra pura (calc), gravação com dedupe por chave + dia, e-mail-resumo por preferência, tela /app/alertas, X c
`test_etapa_f_atribuicao.py`	6	Etapa F — e-mail ao responsável quando a atividade é atribuída (consome activity.assigned).
`test_etapa_f_peca.py`	6	Etapa F — etapa “aguardando peça”: só sinaliza (nunca bloqueia), supervisor+, version, auditoria e selo com texto + ícone na lista, no detal
`test_etapa_f_semanal.py`	4	Etapa F — relatório semanal por e-mail (dono/supervisores) com XLSX, uma vez por semana.
`test_etapa_f_worker.py`	12	Etapa F — worker (ADR-0010): outbox com tentativas/backoff, tarefas agendadas, CLI.
`test_etapa_g_checklist.py`	10	Etapa G (bloco 2) — checklist de inspeção por etapa. Itens no modelo → copiados ao projeto; supervisor+ define itens/“exige”; operador+ pree
`test_etapa_g_evidencias.py`	23	Etapa G (bloco 1) — comentários e fotos por atividade. Regras: só membros da empresa; operador+ comenta/envia; supervisor+ apaga com auditor

Arquivo	Testes	Cobre
`test_etapa_g_storage.py`	15	Etapa G — interface de armazenamento: disco local (dev/test), S3 compatível (SigV4 próprio) e configuração por ambiente (segredos só em env;
`test_etapa_g_unidades.py`	10	Etapa G (bloco 3) — número de série por unidade (rastreabilidade, sem mudar o cálculo).
`test_etapa_h_backup.py`	23	Etapa H — scripts de backup e restauração (scripts/backup_dump.sh, backup_restore_check.sh). Sem PostgreSQL só rodam as conferências de sint
`test_etapa_h_config.py`	30	Etapa H — `flask config-check`, `flask mail-test`, `flask storage-test` e métricas do worker.
`test_etapa_h_render.py`	19	Etapa H — render.yaml, Dockerfile e arquivos de implantação conferem com o app.
`test_etapa_h_saude.py`	6	Etapa H — /saude (liveness do host) e /health (prontidão com banco).
`test_etapa_h_smoke.py`	3	Etapa H — scripts/smoke_check.py contra um servidor local de verdade.
`test_fase4_routes.py`	15	Fase 4 — rotas: limite de plano (402 chamando a rota direto), modo leitura, modelos, novo projeto a partir de modelo, edição, desfazer, impo
`test_fase4_services.py`	35	Fase 4 — serviços: modelos versionados, edição, desfazer, eventos, auditoria, limites de plano, importação e exportação. Rotas ficam em test
`test_fase6.py`	28	Fase 6 — front-end: site público, lista/histórico com filtros, registro via HTMX, resumo, configurações da empresa e assets com hash. Casos
`test_fase7_security.py`	29	Fase 7 — cabeçalhos/CSP, cookies, rate limit, uploads, logs, Sentry, API e MFA.
`test_import_unidade.py`	14	Importação com mapeamento: ajuda por campo e unidade do tempo (ADR-0019 §5). O servidor converte horas em minutos (mesmos limites de sempre)
`test_isolation.py`	65	Isolamento multiempresa: recurso da empresa B acessado por usuário da empresa A → 404. Toda rota com <uuid:…> da área logada precisa de ca
`test_lgpd.py`	16	Fase 7 — LGPD (exportação, exclusão com carência, expurgo, retenção) e acesso de suporte.
`test_migrations.py`	2	Migrations reversíveis e sem drift: upgrade → check → downgrade base → upgrade → check. SQLite (arquivo temporário) sempre; PostgreSQL quand
`test_publico_oliva.py`	10	R10 (ADR-0016): site público na identidade oliva. Páginas, textos-chave, só tokens, CSP e fontes locais.
`test_r3_r5.py`	14	Reestruturação R3–R5: Preencher etapas, modo simples e Meta e equipe / Como funciona. Serviço (projects.py), contas (calc.py) e rotas. Os ca

Arquivo	Testes	Cobre
`test_r3_r5_rotas.py`	19	Reestruturação R3–R5: rotas de Preencher etapas, Meta e equipe e Como funciona. Casos A/B ficam em tests/test_isolation.py (bloco R3–R5).
`test_r6_modulos.py`	28	R6 — módulos opcionais (ADR-0017): desligados por padrão, ligados em Configurações › Módulos. - Desligado: TODA rota do módulo responde 404
`test_r7_planilha.py`	20	R7 — planilha: importar qualquer planilha com mapeamento de colunas; exportar no layout da base (com fórmulas, conferidas contra o calc.py);
`test_r7_varios_projetos.py`	9	R7 — arquivo da equipe com VÁRIOS projetos (abas “Acompanhamento/Equipe/Como funciona” do principal e “<Nome> - Acompanhamento/Equipe/Como f
`test_r8_restaurar.py`	18	R8 — restaurar o projeto para uma data e hora (servidor): serviço e rotas. Casos A/B das rotas novas ficam em tests/test_isolation.py (bloco
`test_rls.py`	15	RLS (ADR-0003): a segunda barreira, conferida com um papel NÃO superusuário. Só roda com TEST_DATABASE_URL em PostgreSQL. O teste cria um pa
`test_schema.py`	18	Schema da Fase 3: regra multiempresa no metadata, constraints, UNIQUE, CHECK e seeds.
`test_seed.py`	1	flask seed-demo: 2 empresas fictícias, 2 projetos (10 e 4 equipamentos) cada, com 3–6 etapas genéricas vindas de um modelo e histórico.
`test_visual.py`	25	Base visual (Fase 2): só tokens, nada de CDN, /estilo só em development, tema e assets.

A contagem é de funções de teste por arquivo; a suíte executa cerca de 1.100 casos porque alguns testes rodam com vários parâmetros.

Documentos de planejamento (`docs/plano/`)

Arquivo	Linhas
`0001-site-renderizado-no-servidor.md`	22
`0002-htmx.md`	22
`0003-postgresql-rls.md`	96
`0004-asaas-paymentprovider.md`	23
`0005-outbox-eventos.md`	29
`0006-hospedagem-render.md`	24
`0007-fase-0-esqueleto.md`	23
`0008-design-sem-claude-design.md`	39

Arquivo	Linhas
`0009-backend-fonte-unica-de-regras.md`	38
`0010-worker-laco-por-empresa.md`	47
`0011-qualidade-fotos-checklist-unidades.md`	61
`0012-deploy-saude-papeis-de-banco.md`	46
`0013-identidade-visual-prumo-noturno.md`	26
`0014-foco-em-etapas.md`	33
`0015-visual-base-controle-de-atividades.md`	67
`0016-identidade-oliva.md`	45
`0017-modulos-opcionais.md`	87
`0018-identidade-oliva-no-sistema.md`	47
`0019-planos-briefing-agendamento.md`	53
`arquitetura.md`	456
`backlog.md`	200
`incidentes.md`	65
`lancamento.md`	278
`lgpd.md`	107
`limpeza-pendente.md`	30
`operacao.md`	410
`pagamentos.md`	210
`pendencias-final.md`	23
`prd.md`	127
`progresso.md`	1429
`reestruturacao.md`	263
`riscos.md`	41
`roadmap.md`	201
`seguranca.md`	158
`worker.md`	75

8. Como o código trabalha (um exemplo completo)

Exemplo: um operador abre “Preencher etapas” e registra que mais 3 peças passaram pela etapa de pintura. Os nomes de arquivo abaixo existem no repositório; a ordem descreve o desenho do sistema.

1. **O navegador envia o formulário** (com um código CSRF que prova que o pedido veio da própria página).
2. **A rota** (em `app/blueprints/preencher.py`) é protegida por `tenant_required` (`app/tenancy.py`): exige login, descobre a empresa pela sessão, confere o papel e, se a empresa estiver em modo leitura por falta de pagamento, recusa com 402. Para rotas de módulos opcionais, `services/modules.py` responde 404 se o módulo estiver desligado.
3. **O serviço** (`app/services/projects.py`) busca a etapa *filtrando pela empresa da sessão*, valida o valor recebido (limites, tipo, versão para evitar dois registros ao mesmo tempo) e grava o registro com o autor.
4. **O cálculo** acontece em `app/services/calc.py`: progresso da etapa, unidades concluídas, tempo restante, dias úteis. Nunca no navegador.
5. **O banco** grava; se o RLS estiver ligado, ele ainda confere que a linha pertence à empresa da conexão.
6. **A auditoria** (`app/services/audit.py`) anota quem fez e quando, sem guardar dado desnecessário.
7. **A resposta** volta como um pedaço de HTML (HTMX) que troca só a parte da página que mudou, com os números calculados pelo servidor.

> **Por que tantas camadas?**
>
> Cada camada pega um erro diferente: o CSRF barra pedidos forjados; o login e o papel barram quem não deveria entrar; o filtro por empresa barra leitura cruzada; o RLS barra erro de programação; a auditoria permite descobrir depois o que aconteceu. Nenhuma delas sozinha é suficiente.

Como o briefing e o agendamento seguem a mesma lógica

- O briefing é dado da empresa (`company_id` + RLS). Só quem está num plano com identidade visual acessa; sem o plano, 404.
- Os horários são dado da plataforma (sem `company_id`); a empresa só enxerga os livres.
- A reserva usa uma restrição única no banco: se duas empresas tentarem ao mesmo tempo, só uma grava.
- O painel `/interno` só abre para conta de equipe com MFA e não lê o conteúdo do briefing.

Vídeos sobre os pontos críticos do código

Estes cobrem os conceitos por trás das decisões mais importantes do projeto. Mesma ressalva da seção 1: não assisti; confira antes.

VÍDEOS: RLS: SEPARAR EMPRESAS NO BANCO (O PONTO MAIS CRÍTICO)

- How To Create Multi-Tenant Apps With Row-Level Security: RLS Series Part 1
  https://www.youtube.com/watch?v=8qPTZQvJ9fA
- PostgreSQL RLS for Safer Multi-Tenant SaaS
  https://www.youtube.com/watch?v=DabJ0hxiXAo
- Row Level Security (RLS) Policy in Postgres
  https://www.youtube.com/watch?v=98yxrZ3Aip4

Documentação oficial (endereço conhecido; não aberto nesta sessão): Manual do PostgreSQL: Row Security Policies

VÍDEOS: MIGRAÇÕES DO BANCO (ALEMBIC / FLASK-MIGRATE)

- SQLAlchemy Migrations Using Flask-Migrate
  https://www.youtube.com/watch?v=uNmWxvvyBGU
- Flask and SQL Fundamentals - Migrations with Flask-Migrate
  https://www.youtube.com/watch?v=_aDEn4n32uY
- How to Handle the Database Migrations with Alembic
  https://www.youtube.com/watch?v=bfelC61XKO4

Documentação oficial (endereço conhecido; não aberto nesta sessão): Alembic

VÍDEOS: BACKUP E RESTAURAÇÃO DO POSTGRESQL

- Backup e Restauração de Banco de Dados PostgreSQL com pg_dump
  https://www.youtube.com/watch?v=ZBWx-CA0jdk
- Backing Up a Postgres Database with pg_dump and pg_restore
  https://www.youtube.com/watch?v=WB6WzuFHcP8
- How to Dump and Restore a Postgres Database Using pg_dump and pg_restore
  https://www.youtube.com/watch?v=bWtsdj68JBo

VÍDEOS: SEGURANÇA WEB: CSP, CSRF E XSS (AS PROTEÇÕES QUE O SISTEMA USA)

- Content Security Policy to prevent XSS attacks
  https://www.youtube.com/watch?v=B0Rz_qiQAWo
- Protect and Defend Against CSRF Attack - OWASP Top 10
  https://www.youtube.com/watch?v=gWMoj9FYTj4
- OWASP Top 10: What is Cross-Site Scripting (XSS)?
  https://www.youtube.com/watch?v=u1rxapgQA2A

Documentação oficial (endereço conhecido; não aberto nesta sessão): OWASP Top 10

VÍDEOS: HTMX COM FLASK (COMO AS PÁGINAS ATUALIZAM SEM RECARREGAR)

- Video Criacao Projeto Flask HTMX (em português)
  https://www.youtube.com/watch?v=OmPmH_3xLo8
- HTMX + Flask: Modern Python Web Apps, Hold the JavaScript
  https://www.youtube.com/watch?v=kNXVFB5eQfo
- Basic CRUD App using HTMX and Flask
  https://www.youtube.com/live/O2Xd6DmcB9g

Documentação oficial (endereço conhecido; não aberto nesta sessão): HTMX

9. Conectores: passo a passo

"Conector" aqui é qualquer serviço externo que o sistema precisa para funcionar em produção. Cada subseção responde a quatro perguntas: **o que é**, **por que precisamos**, **como fazer** e **como saber que deu certo**. Os passos vêm dos arquivos `lancamento.md` e `operacao.md` do repositório; onde o repositório não detalha, eu digo que é o fluxo geral e que você deve confirmar na tela atual do serviço.

> **A regra de ouro dos segredos**
>
> - Chaves e senhas **nunca** vão para chat, e-mail, print, código ou commit. Elas vão só para o gerenciador de senhas e para os campos de variáveis do Render.
> - Staging (ensaio) e produção usam **chaves diferentes**. Se uma vazar, a outra continua segura.
> - Se você me mostrar uma tela com uma chave visível, eu não a guardo; troque a chave depois.

9.0 Antes de tudo: cofre de senhas, e-mail do negócio e 2FA VOCÊ FAZ

O que é: um aplicativo que guarda todas as senhas (Bitwarden, 1Password) e um e-mail só do negócio. **Por que:** são pelo menos dez contas (GitHub, Render, Cloudflare, Resend, Sentry, Asaas, Registro.br, monitor). Se uma for invadida e você usar a mesma senha em outra, o dano se espalha. O 2FA (código no celular) impede que só a senha seja suficiente.

1. Crie um e-mail só para o negócio (ex.: no próprio domínio, depois; até lá, um e-mail novo).
2. Instale um gerenciador de senhas e ligue o 2FA nele primeiro.
3. Em **cada** conta que criar daqui para frente: senha gerada pelo cofre + 2FA ligado + códigos de recuperação guardados no cofre.
4. Crie uma pasta "Controle de Produção" no cofre, com subitens "Staging" e "Produção".

VÍDEOS: GERENCIADOR DE SENHAS E 2FA

- How To Turn on 2 Step Verification in Bitwarden
  https://www.youtube.com/watch?v=V5uge64ZN7U
- how to enable two factor authentication in bitwarden full guide 2026
  https://www.youtube.com/watch?v=qHU1rUfQSb0

Documentação oficial (endereço conhecido; não aberto nesta sessão): Ajuda do Bitwarden

9.1 Domínio: Registro.br VOCÊ FAZ

O que é: o registro do nome do site (ex.: `controleproducao.com.br`; o nome é só um exemplo do plano, a disponibilidade não foi verificada). **Por que:** o domínio é a base do endereço do site, do certificado HTTPS e dos e-mails (SPF/DKIM/DMARC). Sem ele não há lançamento.

1. Entre em registro.br e crie a conta com seu CPF ou CNPJ. Ligue o 2FA.
2. Pesquise o nome desejado e registre. Ligue a **renovação automática**.
3. Decida o **endereço principal**. Recomendação do plano: `www.seudominio.com.br`, com o endereço sem "www" redirecionando para ele. Motivo técnico: o cookie de sessão do sistema é por endereço, então um

único endereço deve servir o aplicativo.

4. Anote o endereço principal por escrito: ele vira o valor de `APP_BASE_URL`.
5. Se for usar a Cloudflare para o DNS (recomendado, ver 9.2), você troca os servidores DNS do domínio no Registro.br pelos dois que a Cloudflare indicar.

Como saber que deu certo: o domínio aparece ativo no painel do Registro.br, com renovação automática ligada, e o endereço principal está definido por escrito.

VÍDEOS: REGISTRAR DOMÍNIO .COM.BR

- Como Registrar Domínio no Registro.br - Atualizado 2026
  https://www.youtube.com/watch?v=zPXIVWTAFGU
- Como REGISTRAR UM DOMÍNIO .COM.BR para Seu Negócio em 3 MINUTOS
  https://www.youtube.com/watch?v=jZmquG4fmNQ
- Como REGISTRAR um DOMÍNIO no REGISTRO.BR e CONFIGURAR DNS na HOSPEDAGEM (Passo a Passo 2025)
  https://www.youtube.com/watch?v=WhtAQVTb_DI
- Apontamento de DNS na Registro.br e Provedor de Serviços (2024)
  https://www.youtube.com/watch?v=TCCYDZaRJ80

Documentação oficial (endereço conhecido; não aberto nesta sessão): Registro.br

Os valores de anuidade citados em alguns vídeos são antigos (um deles é de 2022). Confira o preço atual no próprio Registro.br.

9.2 DNS: Cloudflare VOCÊ FAZ

O que é: o "catálogo de endereços" do seu domínio. Diz para onde vai o site (Render), qual servidor envia e-mail (Resend) e assim por diante. **Por que a Cloudflare:** ela entrega o DNS gratuito e também o R2 (9.3) na mesma conta. A alternativa é manter o DNS no Registro.br; os registros são os mesmos.

1. Crie a conta na Cloudflare (e-mail do negócio, senha do cofre, 2FA).
2. Adicione o domínio ("Add a site") e escolha o plano gratuito.
3. A Cloudflare mostra dois servidores de nomes (nameservers). Copie os dois.
4. No Registro.br, troque os servidores DNS do domínio por esses dois. A propagação pode demorar; a Cloudflare avisa quando o domínio fica ativo.
5. Depois, nesta mesma tela de DNS, você vai criar os registros do Render (CNAME do site) e do Resend (DKIM, SPF, DMARC).

Como saber que deu certo: a Cloudflare marca o domínio como ativo; consultas `dig` ao domínio respondem pelos servidores da Cloudflare.

VÍDEOS: CLOUDFLARE E NAMESERVERS

- How to Add Nameservers in Cloudflare [2026 Guide]
  https://www.youtube.com/watch?v=yEEnyOKiZF0
- How to Change Nameservers in Cloudflare (2026) Step-by-Step Full Guide
  https://www.youtube.com/watch?v=tPLLPdFLQ68
- HOW TO ADD DOMAIN TO CLOUDFLARE (STEP-BY-STEP GUIDE 2026)
  https://www.youtube.com/watch?v=uTbWRgCSt18
- How to Point Your Domain to Cloudflare
  https://www.youtube.com/watch?v=XgfRHdFjD_Y

Documentação oficial (endereço conhecido; não aberto nesta sessão): Cloudflare

Estes vídeos são em inglês e usam outros registradores como exemplo; a parte que importa (copiar os dois nameservers e trocá-los no registrador) é a mesma.

9.3 Armazenamento de arquivos: Cloudflare R2 VOCÊ FAZ

O que é: um "armário" de arquivos compatível com o S3 da Amazon. **Por que:** o sistema guarda fotos de evidência e do briefing, e as cópias de segurança do banco. Nunca em pasta pública: só por rota autenticada. Três buckets privados, separados por finalidade e ambiente.

1. Na Cloudflare, abra R2 e crie os buckets **privados**: `controle-fotos`, `controle-fotos-staging` e `controle-backup`.
2. Crie dois tokens de API do R2 com o menor acesso possível: (1) **app**: leitura e escrita só no bucket de fotos de cada ambiente; (2) **backup**: leitura e escrita em `controle-backup` e *só leitura* em `controle-fotos`.
3. No `controle-backup`, crie uma regra de ciclo de vida que expire os arquivos da pasta `db/` em 35 dias (o sistema guarda 30 dias).
4. Do painel, anote: o endereço S3 (algo como `https://<id-da-conta>.r2.cloudflarestorage.com`, confira na tela), o nome do bucket, o Access Key ID e o Secret Access Key. Eles vão para as variáveis `STORAGE_S3_*` (app) e `BACKUP_S3_*` (cron de backup). O Secret só aparece uma vez: guarde no cofre.
5. No Render, preencha as variáveis e rode `flask storage-test` no Shell do serviço web.

Como saber que deu certo: `flask storage-test` responde "Armazenamento OK"; uma foto enviada numa atividade aparece, e o endereço dela só abre com login.

> **Por que dois tokens**
> Se o token do app vazar, o atacante lê/escreve fotos, mas não toca nos backups. Se o do backup vazar, ele não consegue escrever no bucket de fotos.

VÍDEOS: R2

- Cloudflare R2 Object Storage Tutorial: Create Bucket & Custom Domain Setup
  https://www.youtube.com/watch?v=ZvczexjXt7E
- R2 - S3 API compatibility
  https://www.youtube.com/watch?v=ohfg-lCt6hc
- How to Upload Files to Cloudflare R2 (2026 Updated Guide)
  https://www.youtube.com/watch?v=pQmGBSRTdPQ

Documentação oficial (endereço conhecido; não aberto nesta sessão): Documentação do R2

Nenhum desses vídeos mostra, pelo que a busca devolveu, a criação do token de API passo a passo; para isso use a documentação oficial acima.

9.4 Hospedagem: Render (site, worker, crons, banco e Redis) VOCÊ FAZ

O que é: o lugar onde o sistema roda. **Por que o Render:** escolha registrada no ADR-0006; o repositório já traz os arquivos de configuração (`render.staging.yaml` e `render.yaml`) gerados por script e conferidos pelo CI.

O que você vai criar

Item	Nome	Função	Plano gratuito?
PostgreSQL 16	controle-db	Todos os dados das empresas	Sim, mas **expira em 30 dias** (14 dias de tolerância; depois o Render apaga) e tem 1 GB
Key Value (Redis)	controle-redis	Limite de tentativas e fila	Existe; confirme no painel
Site (web)	controle-web	O sistema que o cliente usa	Sim, mas dorme após 15 min sem acesso e demora ~1 min para acordar
Worker	controle-worker	Alertas, relatórios, e-mails em fila	**Não** (a partir do plano Starter)
Cron lgpd	controle-lgpd	Exclusões e retenção, todo dia às 03:00 (Brasília)	**Não**; cobra enquanto roda, mínimo US$ 1/mês por job
Cron backup	controle-backup	Backup criptografado, todo dia às 04:00 (Brasília)	**Não**
Cron monitor	controle-monitor	Checa a fila a cada 10 minutos	**Não**

Fonte dos limites do plano gratuito: Deploy for Free (Render), expiração de 30 dias do PostgreSQL gratuito e Services and Service Types. Preços (a partir de cerca de US$ 6/mês para o banco pago) são os que a própria Render publica; confirme o valor atual no painel.

Região: por que “Virgínia” e o que isso significa

A Virgínia é um data center nos EUA. As regiões do Render são Oregon, Ohio, Virgínia, Frankfurt e Singapura: **não há região no Brasil**. Virgínia é a mais próxima de São Paulo, com latência de ida e volta na casa de 100 a 200 ms acima do que seria no Brasil (estimativa; não medida). Os dados dos clientes ficam nos EUA: isso é

transferência internacional de dados pela LGPD e deve ser tratado com o advogado (9.11). DECISÃO SUA

Passo a passo (staging primeiro)

1. Entre no Render com o e-mail do negócio e 2FA. O workspace já existe ("My Workspace") e está vazio.
2. Conecte o GitHub ao Render e autorize o repositório `pecordeiro/saas`.
3. Crie no GitHub a branch `staging` (o `render.staging.yaml` faz deploy a partir dela).
4. Crie o **PostgreSQL 16** de staging, região Virgínia. Nome sugerido: `controle-db-staging`. No ensaio pode ser o plano gratuito; anote a data de criação (expira em 30 dias) e passe para um plano pago antes de colocar qualquer dado real.
5. Crie o **Key Value** de staging: `controle-redis-staging`.
6. (Opcional, recomendado) Instale o Render CLI e rode `render blueprints validate render.staging.yaml`. Confirme os nomes de plano e o custo mensal antes de aprovar.
7. No Render: *New* → *Blueprint* → o repositório → arquivo `render.staging.yaml`. Ele cria o site e o worker de staging, ligados ao banco e ao Redis.
8. O Render pergunta, uma única vez, os valores das variáveis marcadas como segredo (tabela 9.12). Cole direto ali.
9. O primeiro deploy roda automaticamente `flask config-check --online`, `flask db upgrade` (cria as tabelas) e `flask seed-plans` (cadastra os planos). Se o `config-check` reprovar, o deploy para: é proposital.
10. Adicione o domínio de staging (ex.: `staging.seudominio.com.br`) no serviço web e crie o registro CNAME no DNS (9.2).
11. Abra `https://staging…/saude`: deve responder 200. Rode `python scripts/smoke_check.py https://staging…` (0 falhas esperadas).

Produção (depois do staging passar)

1. Mesmo processo com `render.yaml` (branch `main`) e segredos **de produção** (Asaas de produção, bucket de produção).
2. Domínio do site: o endereço principal (www) e o sem "www" redirecionando.
3. Depois do primeiro deploy, no Shell do web: `flask seed-holidays --de 2026 --ate 2035`; só então `RLS_ENABLED=1 flask rls enable`; conferir `flask rls status` (todas as tabelas "ligado=sim forçado=sim") e `flask config-check --online --strict` (APROVADO). **A ordem importa**: o seed de feriados não funciona depois que o RLS é forçado.

O ponto mais crítico: o papel do banco (P-14)

O RLS só protege se o usuário do banco que o site usa **não** for superusuário nem tiver `BYPASSRLS`. No Render, o usuário padrão do banco costuma ser o dono; o `config-check --online` acusa isso. Se acusar, **pare e me chame**: a solução é criar um papel comum para o app, o worker e os crons, e deixar o papel dono só para migrações e backup. Isso muda as variáveis de conexão no `render.yaml` e exige um ADR com a sua aprovação (descrito em `operacao.md` seção 4 e no ADR-0012).

Como saber que deu certo: deploy verde, `/saude` = 200, `/health` = 200 (banco respondendo), `flask rls status` sem "não".

Estado hoje: o Render está conectado, mas sem nenhum serviço ou banco. Uma tentativa minha de criar o PostgreSQL de staging foi bloqueada pelo sistema de permissões ("modificar recursos compartilhados"); nada foi criado e não contornei o bloqueio.

VÍDEOS: RENDER, FLASK E POSTGRESQL

- How to Deploy a Flask App and Postgres Database to Render
  https://www.youtube.com/watch?v=IBfj_0Zf2Mo
- How to Deploy a Flask App and Postgres Database to Render for FREE
  https://www.youtube.com/watch?v=iH0XlesfwU4
- Deploy a Dockerized App on Render with PostgreSQL Database FOR FREE
  https://www.youtube.com/watch?v=_IdejOUafe4

Documentação oficial (endereço conhecido; não aberto nesta sessão): Documentação do Render

O sistema usa Docker e Blueprint (arquivo YAML), então o terceiro vídeo é o mais próximo; os dois primeiros ajudam a entender a tela do Render. São de anos diferentes: a tela mudou.

9.5 Código e verificação automática: GitHub VOCÊ FAZ

O que é: onde o código fica guardado e versionado (repositório `pecordeiro/saas`). **Por que:** o Render faz deploy a partir dele, e o GitHub Actions (`.github/workflows/ci.yml`) roda, a cada envio, a auditoria de dependências, os testes em SQLite *e em PostgreSQL 16*, as migrações e o build do Docker.

1. Ligue o 2FA na sua conta do GitHub.
2. Crie as branches `staging` e `main` a partir da branch de desenvolvimento quando for o momento do ensaio e da produção.
3. Abra um pull request da branch de desenvolvimento e espere o CI. **O PostgreSQL só é testado de verdade aqui** (P-11).
4. Se algum job falhar, me mande o log; eu corrijo.

Como saber que deu certo: todos os jobs verdes (audit, test, docker).

Nota: o conector do GitHub desta sessão não conectou; por isso não consegui ver o resultado do CI daqui.

9.6 E-mail do sistema: Resend VOCÊ FAZ

O que é: serviço que envia os e-mails (confirmação de conta, convites, redefinição de senha, avisos, relatório semanal). **Por que:** e-mail enviado sem autenticação (SPF, DKIM, DMARC) cai no spam. Sem a chave, o sistema entra em modo "console" (nada é entregue) e o `config-check` reprova em produção.

1. Crie a conta no Resend (e-mail do negócio, 2FA).
2. Use um **subdomínio só para envio** (ex.: `mail.seudominio.com.br`): isso isola a reputação do domínio principal.
3. Resend → Domains → Add Domain → `mail.seudominio.com.br`.
4. O Resend mostra os registros exatos. Em geral são: um `TXT` de DKIM (`resend._domainkey.mail`), um `MX` e um `TXT` de SPF em `send.mail`. Crie cada um na Cloudflare (9.2), copiando os valores da tela do Resend.
5. Crie o DMARC (o Resend não cria): um `TXT` em `_dmarc.seudominio.com.br` com o valor `v=DMARC1; p=none; rua=mailto:dmarc@seudominio.com.br; fo=1; adkim=r; aspf=r`. Comece só observando

(`p=none`); depois de 2 a 4 semanas sem falhas legítimas, suba para `quarantine` e depois `reject`. O endereço `rua` precisa ser uma caixa que você lê.

6. Se o domínio já tiver um registro SPF para outro serviço, **não crie um segundo**: só vale um por nome. Mescle com `include:`.
7. Espere o status "Verified". Gere a chave de API com permissão só de envio nesse domínio.
8. No Render, preencha `RESEND_API_KEY`, `MAIL_FROM` (ex.: `"Controle de Produção <nao-responda@mail.seudominio.com.br>"`) e `CONTACT_EMAIL`.
9. No Shell do web: `flask mail-test voce@gmail.com`.

Como saber que deu certo: o e-mail chega na caixa de entrada (não no spam). Em "Mostrar original", procure `Authentication-Results` com `spf=pass`, `dkim=pass` e `dmarc=pass`. Repita para Gmail, Outlook e um e-mail corporativo. O mail-tester.com deve dar 9 ou mais de 10.

Depois do lançamento: taxa de rejeição (bounce) abaixo de 2% e de reclamações abaixo de 0,1%. Se subir, pause envios em massa e investigue.

VÍDEOS: RESEND, SPF, DKIM E DMARC

- How to Verify a Domain in Resend
  https://www.youtube.com/watch?v=CF6gRy-9_LE
- Send E-Mails with resend tutorial Get API & Domain Verification
  https://www.youtube.com/watch?v=pHY8nzgfz4M
- Getting Started with Resend
  https://www.youtube.com/watch?v=FzoakVwRm2Q
- DNS PARA E-MAILS - Como configurar no Registro.br (DKIM, SPF, MX e DMARC)
  https://www.youtube.com/watch?v=vR9MrAMCbqM
- O que é e como configurar o DMARC
  https://www.youtube.com/watch?v=PufbDpElnpc

Documentação oficial (endereço conhecido; não aberto nesta sessão): Resend

9.7 Cobrança: Asaas VOCÊ FAZ

O que é: plataforma de cobrança (Pix, boleto, cartão) com assinaturas recorrentes. **Por que:** é como as empresas pagam o plano; o sistema cria a assinatura, recebe avisos (webhooks) quando alguém paga ou atrasa e muda o acesso: ativo, atrasado ou modo leitura.

> **Atenção: a integração nunca falou com o Asaas de verdade**
>
> O documento `pagamentos.md` registra que o acesso à documentação do Asaas estava bloqueado na rede onde a integração foi escrita. Ela foi construída com um provedor simulado e testes. **A homologação no sandbox (15 passos em `pagamentos.md` seção 5) é o primeiro teste real.** Divergências viram ajustes de código, que eu faço.

1. Crie a conta de **produção** no Asaas e envie os documentos (cadastro com verificação; pode levar dias, por isso comece já).
2. Crie também a conta de **sandbox** (ambiente de testes, sem dinheiro real). Chaves de sandbox começam com `$aact_hmlg_`; as de produção com `$aact_prod_`.

3. No sandbox, gere a chave de API e guarde no cofre. Defina um token de webhook (texto longo e aleatório; comandos na seção 9.10).
4. Em Integrações → Webhooks, cadastre a URL `https://staging…/webhooks/asaas`, o token (o Asaas o envia no cabeçalho `asaas-access-token`), API v3, e os eventos de cobrança e de assinatura.
5. No Render (staging), preencha `ASAAS_API_KEY` e `ASAAS_WEBHOOK_TOKEN`. A `ASAAS_BASE_URL` de sandbox já vem no YAML.
6. Execute os 15 passos de `pagamentos.md` seção 5 (criar assinatura, pagar no sandbox, atrasar, estornar, trocar de plano…), registrando data e observações. Rode `flask billing-reconcile` duas vezes; na segunda, "0 corrigidas".
7. Em produção: mesma configuração com a URL de produção, o token de produção e a chave `$aact_prod_…`; depois faça uma **cobrança real de valor baixo**, pague por Pix, confira a assinatura "Ativa", estorne no Asaas e confira o modo leitura.
8. Combine com o contador a emissão de nota fiscal de cada mensalidade (9.11) **antes da primeira cobrança real**.

Como saber que deu certo: pagamento vira assinatura "Ativa"; estorno vira modo leitura; o webhook aparece como entregue no Asaas; a segunda reconciliação não corrige nada.

VÍDEOS: ASAAS (API, SANDBOX, WEBHOOKS, ASSINATURAS)

- API do Asaas: como configurar sua conta no Sandbox?
  https://www.youtube.com/watch?v=VSfi3dNI9RY
- API do Asaas: tudo sobre Webhooks, tipos de envio e principais eventos
  https://www.youtube.com/watch?v=Yk6yTfaLVDY
- Assinaturas na API do Asaas: automatize cobranças recorrentes
  https://www.youtube.com/watch?v=m-ahYNfK_UU
- API do Asaas: como funciona a penalização de filas de webhooks?
  https://www.youtube.com/watch?v=q3hZQzLx4XM

Documentação oficial (endereço conhecido; não aberto nesta sessão): Documentação do Asaas

O quarto vídeo é importante: se o nosso site ficar fora do ar ou responder erro, o Asaas pode pausar a fila de webhooks; o monitor da seção 9.9 existe também por isso.

9.8 Erros: Sentry VOCÊ FAZ

O que é: serviço que recebe os erros do sistema e avisa você. **Por que:** sem ele, você só descobre um erro quando o cliente reclama. **O que já está pronto no código:** o envio é filtrado (sem cookies, senhas, tokens, e-mails ou IP; só o identificador do usuário). Isso foi coberto por testes.

1. Crie a conta e um projeto (plataforma Python/Flask). Copie o DSN (endereço de envio).
2. No projeto, configure: retenção de 90 dias, Data Scrubbing ligado, "Prevent storing of IP addresses" ligado, alertas de "novo issue" e "issue regrediu" por e-mail.
3. No Render, preencha `SENTRY_DSN`. Mantenha `SENTRY_TRACES_SAMPLE_RATE=0` (sem medição de desempenho por padrão).
4. No Shell do web: `flask sentry-test`. Abra o evento no Sentry.

Como saber que deu certo: no evento, o campo `token` aparece como "[filtrado]", o e-mail como "[e-mail]" e o usuário só tem `id`.

VÍDEOS: SENTRY

- Aprenda a monitorar o erros das suas aplicações com o Sentry
  https://www.youtube.com/watch?v=5OTXem5LFTI
- Using Sentry With Flask to Log Errors
  https://www.youtube.com/watch?v=zVW83QUSDdQ

Documentação oficial (endereço conhecido; não aberto nesta sessão): Documentação do Sentry

9.9 Disponibilidade e logs: monitor de uptime e retenção de logs VOCÊ FAZ

O que é: um serviço que visita o site a cada 1 a 5 minutos e avisa no seu celular se ele cair. **Por que:** o monitor interno só olha a fila; quem avisa que o site inteiro caiu é um observador de fora. Os logs do Render duram pouco; a política de privacidade promete 6 meses.

Monitor	Endereço	Espera	Para quê
Processo no ar	`/saude`	200 com "status: ok"	O programa está de pé
Banco respondendo	`/health`	200 (503 = banco fora)	O site consegue falar com o banco
Página pública	o domínio principal	200 + uma palavra da página	HTTPS e roteamento
Login	`/entrar`	200	Rota pública crítica
Certificado	os mesmos endereços	validade maior que 14 dias	Avisar antes de vencer

1. Crie a conta num monitor (UptimeRobot, Better Stack…) e cadastre os monitores acima. Alerta por e-mail **e** por mensagem no celular.
2. Use a página de status do próprio monitor no começo.
3. No Render, confirme em Notificações que "cron job failed" está ligado (é assim que você é avisado se o backup ou o monitor falhar).
4. No Render → Workspace → Log Streams, envie os logs para um provedor com retenção de pelo menos 6 meses (Better Stack, Papertrail, Datadog… confirme o custo).
5. **Teste de verdade:** derrube o staging de propósito por 2 minutos (suspenda o serviço) e confirme que o alerta chegou no celular.

Como saber que deu certo: o alerta de teste chegou. Disponibilidade da página no painel do monitor acima de 99,5% em 30 dias é a meta de atenção.

VÍDEOS: UPTIMEROBOT

- Como Descobrir se Meu Site Está Fora do Ar Com UptimeRobot [Ferramenta Gratuita]
  https://www.youtube.com/watch?v=ofXuyWbt9aY
- Monitorando sua aplicação de forma simples com UptimeRobot
  https://www.youtube.com/watch?v=DszgYLBhrs0

Documentação oficial (endereço conhecido; não aberto nesta sessão): UptimeRobot

Os limites do plano gratuito do UptimeRobot citados em vídeos de terceiros podem estar desatualizados; confirme no site.

9.10 Gerar os segredos e as chaves de backup VOCÊ FAZ

Por que: a chave da aplicação assina os cookies de login; a de MFA criptografa os segredos do código no celular; as chaves de backup criptografam e assinam as cópias do banco. Devem ser diferentes entre staging e produção.

```
python -c "import secrets; print(secrets.token_hex(32))"      # SECRET_KEY
python -c "import secrets; print(secrets.token_hex(32))"      # MFA_ENCRYPTION_KEY (outra!)
python -c "import secrets; print(secrets.token_urlsafe(32))"  # ASAAS_WEBHOOK_TOKEN
age-keygen -o chave-backup.txt                                # criptografia do backup
ssh-keygen -t ed25519 -N "" -C controle-backup -f assina-backup   # assinatura do backup
```

1. Rode os comandos no seu computador (precisa de Python; `age` e `ssh-keygen` para as duas últimas).
2. Guarde **tudo** no cofre. Faça uma **cópia offline** da `chave-backup.txt` (privada) e da `assina-backup.pub`.
3. Só o conteúdo de `assina-backup` (privada) vai para o Render, na variável `BACKUP_SIGN_KEY` do cron de backup. A chave pública do age vai em `BACKUP_AGE_RECIPIENT`.
4. Nada disso vai para chat, e-mail ou commit.

> **Sem a chave privada do backup, o backup é inútil**
> Quem tem `chave-backup.txt` lê todas as cópias; quem perde essa chave não consegue restaurar nada. Por isso a cópia offline é obrigatória.

9.11 Profissionais: contador e advogado VOCÊ FAZ

Quem	O que levar	O que perguntar / pedir	Quando
Contador	`pagamentos.md` (seções 1 e 2) e os preços novos	Quem emite a NFS-e de cada mensalidade (manual ou automação do Asaas); CNAE e regime; retenções; como tratar estorno; plano "Implantação" sob orçamento.	**Antes da primeira cobrança real**
Advogado	`docs/legal/termos.md`, `privacidade.md`, `lgpd.md`, `incidentes.md`	Nomear o encarregado de dados (DPO) e o e-mail; **transferência internacional** (Render/EUA, Resend, Sentry, Cloudflare); janela de backups (30 dias + janela do Render); contrato de operador (você é operador das empresas clientes); foro e limitação de responsabilidade; carência de 7 dias e modo leitura.	Comece cedo: demora

Depois que o advogado aprovar, me passe o texto final: eu troco as páginas e removo o aviso de "Rascunho". O verificador `smoke_check.py --producao` confere se ainda sobra algum `[colchete]`.

VÍDEOS: LGPD (APOIO PARA A CONVERSA; NÃO SUBSTITUI O ADVOGADO)

- Lei Geral de Proteção de Dados (LGPD) no Software como Serviço (SaaS)
  https://www.youtube.com/watch?v=5cDydr-rjzg
- Impactos da LGPD no Desenvolvimento de Software
  https://www.youtube.com/watch?v=xb5WS47nmCY
- LGPD: resumo da lei em 4 minutos
  https://www.youtube.com/watch?v=n3e0HVcNmI0

9.12 Quadro das variáveis: o que preencher no Render e de onde vem

Estas são as variáveis marcadas como segredo (`sync: false`) no `render.staging.yaml`. O Render pergunta o valor uma vez, no serviço web. O que não está aqui (como `APP_ENV`, `RLS_ENABLED`, `BILLING_PROVIDER`) já vem preenchido no YAML.

Variável	Para que serve	De onde vem
`SECRET_KEY`	Assina cookies de login	Você gera (9.10)
`MFA_ENCRYPTION_KEY`	Criptografa os segredos do MFA	Você gera (9.10); diferente da anterior
`APP_BASE_URL`	Endereço do site (links de e-mail)	Endereço principal do domínio (9.1)
`CONTACT_EMAIL`	E-mail de atendimento exibido no site	Caixa do domínio
`MAIL_FROM`	Remetente dos e-mails	Domínio verificado no Resend (9.6)
`RESEND_API_KEY`	Envio de e-mail	Painel do Resend (9.6)
`ASAAS_API_KEY`	Chamadas à API do Asaas	Painel do Asaas (9.7); sandbox em staging
`ASAAS_WEBHOOK_TOKEN`	Prova que o aviso veio do Asaas	Você gera (9.10) e cadastra no Asaas
`STORAGE_S3_ENDPOINT / BUCKET / ACCESS_KEY_ID / SECRET_ACCESS_KEY`	Fotos no R2	Painel da Cloudflare R2 (9.3)
`SENTRY_DSN`	Envio de erros	Painel do Sentry (9.8)
`STAFF_NOTIFY_EMAIL`	Para onde vão os avisos de briefing e de ligação	Seu e-mail de equipe. Se faltar, o `config-check` só avisa
`DATABASE_URL`, `REDIS_URL`	Conexão com o banco e o Redis	Automático: o Render preenche a partir do banco e do Key Value
`BACKUP_*`, `RESTORE_ADMIN_URL`	Cron de backup e teste de restauração	Chaves da 9.10 e R2 da 9.3 (só no cron de backup)

10. Ordem recomendada para configurar tudo

A ordem importa porque algumas coisas dependem de outras (e-mail depende do domínio; o site depende do banco). Os itens de prazo longo (cadastros com documento, advogado, contador) devem começar no primeiro dia, em paralelo.

Bloco	O que fazer	Depende de	Pronto quando
A. Hoje, em paralelo	Cofre de senhas e 2FA (9.0). Domínio (9.1). Abrir contas: Render, Cloudflare, Resend, Sentry, Asaas (produção e sandbox), monitor. Marcar contador e advogado (9.11).	—	Todas as contas logam com 2FA; domínio ativo; Asaas em análise
B. Chaves	Gerar os segredos e as chaves de backup (9.10).	A	Tudo no cofre; cópia offline da chave de backup
C. DNS e arquivos	Cloudflare DNS (9.2) e R2 (9.3).	A (domínio)	Domínio ativo na Cloudflare; buckets e tokens criados
D. E-mail	Resend: domínio, DNS, chave (9.6).	C	"Verified"; `mail-test` com SPF/DKIM/DMARC passando
E. Staging	Banco, Redis, web e worker (9.4); variáveis; domínio de staging.	B, C, D	`/saude` = 200; `smoke_check` sem falhas
F. Ensaio	Asaas sandbox (9.7); `storage-test`, `sentry-test`; conferência visual em 375, 768 e 1280 px; CI verde (9.5).	E	Os 15 passos do Asaas ok; CI verde
G. Papel do banco	Checar o papel do banco e decidir o RLS (P-14).	E	Papel comum (sem superusuário e sem BYPASSRLS) ou solução com dois papéis testada
H. Produção	Blueprint de produção; domínio; feriados; RLS; conta de equipe com MFA; webhook de produção; cobrança real de valor baixo; isolamento A/B com duas empresas de teste.	F, G e jurídico/contábil	Roteiro do smoke test da Parte C do `lancamento.md` sem falha
I. Rede de proteção	Backup real e **teste de restauração**; monitor de uptime com alerta testado; logs com 6 meses de retenção.	H	"RESULTADO: OK" na restauração; alerta de teste recebido
J. Piloto	Convidar o primeiro cliente piloto; canal de suporte; avisar que é piloto.	I	O piloto consegue cadastrar, criar modelo, projeto e registrar peças sem ajuda

> **Regra do roadmap**
>
> Se faltar isolamento entre empresas, segredos, backup ou login em rota privada, **não lance**.

11. Como conferir que deu certo

Comandos para rodar no "Shell" do serviço no Render (ou localmente, com as variáveis do ambiente). Todos estão implementados no repositório.

Comando	O que confere
`flask config-check --online --strict`	Configurações inseguras, papel do banco, migrações, RLS e feriados. Nunca imprime valores de segredos.
`python scripts/smoke_check.py https://staging…`	Teste sem login: páginas públicas respondem, rotas privadas exigem login, sem "[colchetes]" nem aviso de rascunho (com `--producao`).
`flask mail-test voce@gmail.com`	Envio real de e-mail; mostra backend, domínio do remetente e destino mascarado.
`flask storage-test`	Escrita e leitura no R2.
`flask sentry-test`	Evento de erro com dados falsos; confira o filtro no Sentry.
`flask worker status --check`	Fila do worker (sai com erro se há evento esgotado ou parado há mais de 15 min).
`flask rls status`	Se cada tabela tem RLS ligado e forçado.
`flask billing-reconcile`	Corrige divergências entre o Asaas e o banco; rode duas vezes.
`flask staff conceder … --motivo … --autorizado-por …`	Dá acesso de equipe (painel `/interno`); exige e-mail confirmado e MFA; fica auditado.
`scripts/backup_restore_check.sh`	Baixa o backup, confere assinatura e restaura num banco temporário; mede o tempo (insumo do RTO).

Roteiro de aceite (resumo)

1. Abra o site pelo domínio: HTTPS válido e redirecionamento do endereço sem "www".
2. Cadastre-se, confirme o e-mail (a mensagem deve chegar na caixa de entrada).
3. Crie um modelo, um projeto, registre peças. Confira o painel.
4. Crie duas empresas de teste (A e B): nenhum dado de A aparece para B, e endereços de B abertos por A dão 404.
5. Assine um plano no sandbox, pague, atrase, estorne. Confira o acesso a cada passo.
6. Suba uma foto; abra o endereço dela deslogado (deve pedir login).
7. Derrube o staging por 2 minutos: o alerta deve chegar no celular.
8. Restaure um backup num banco temporário e confira.

O roteiro completo, com identificadores (C1 a C10), está na Parte C de `docs/plano/lancamento.md`; o plano de recuo (rollback) está na Parte D e o critério de liberar ou não (go/no-go) na Parte E.

12. Tudo que fica em aberto

Esta seção reúne tudo que ainda não está feito, não está decidido ou não foi verificado. Foi montada a partir de `docs/plano/progresso.md`, `pendencias-final.md`, `lancamento.md` e do que conversamos.

12.1 Decisões que são suas DECISÃO SUA

Decisão	Situação / padrão que adotei	Por que importa
Teste grátis de 14 dias nos planos 1 e 2	Mantive. Você não respondeu.	Muda a cobrança no cadastro
PostgreSQL para testar o isolamento	Aguardando: você me passa uma URL de banco de teste (vazio, sem dados reais) *ou* autoriza que eu suba um local	É a maior dúvida técnica: o RLS só foi provado em SQLite na branch principal
Região do Render	Virgínia (EUA), a mais próxima do Brasil entre as oferecidas	Transferência internacional de dados (LGPD) e latência
Acesso do profissional / e-mail de auditoria (plano Implantação)	Fora desta entrega; você abrirá o e-mail depois	Define o que ele pode ler, por quanto tempo e com MFA. Exige um ADR novo
A equipe pode ler o conteúdo do briefing?	Não nesta entrega: o painel mostra só status e horários	Leitura entre empresas é exceção ao isolamento e precisa de regra clara
Formulário de contato no site	Não existe: `/contato` só mostra o e-mail	Se for criar, precisa de limite por IP, campo armadilha e CSRF; exige ADR
API pública no lançamento?	Desligada por padrão; só 2 rotas de leitura prontas, 3 respondem “não implementado”	Implementar escrita aumenta a superfície de ataque
Fotos reais da página inicial	Aguardando as suas fotos; nada foi alterado	Credibilidade da página de abertura
Imagem de compartilhamento (og-image) e nomes de exemplo da faixa de produtos	Aguardando a sua validação	Aparece em links compartilhados e na página inicial
Regra de “etapa atrasada” e “faltam” em cinza	Tratados como aprovados por padrão	Muda o que aparece no painel
Planos pagos do Render e custo mensal	Aguardando: o repositório usa “starter” em todos os serviços	Custo recorrente
Remarcação de ligação	Decidido: o cliente pede, você decide	—
Supabase	Decidido manter Render + PostgreSQL	Trocar exigiria ADR novo e retrabalho

12.2 O que só você consegue fazer (checklist de lançamento) VOCÊ FAZ

Item	O que é	Seção do guia
P-01	Registrar o domínio e definir o endereço principal	9.1
P-02	Contador: nota fiscal e enquadramento (antes da 1ª cobrança real)	9.11
P-03	Advogado: Termos, Privacidade, encarregado (DPO), transferência internacional	9.11
P-04	Criar as contas de produção (Render, Resend, Cloudflare, Sentry, Asaas, monitor, caixa de e-mail do domínio)	9.0 a 9.9
P-05	Gerar e guardar segredos e chaves de backup	9.10
P-06	Cloudflare: DNS e R2 (buckets e tokens)	9.2, 9.3
P-07	Resend: domínio e SPF/DKIM/DMARC, teste de entrega	9.6
P-08	Subir o staging no Render	9.4
P-09	Homologar o Asaas no sandbox (15 passos)	9.7
P-10	Validar os Blueprints (Render CLI) e aprovar o custo	9.4
P-11	CI verde, incluindo a suíte em PostgreSQL 16	9.5
P-12	Testes de e-mail, fotos e Sentry no staging	9.3, 9.6, 9.8
P-13	Conferência visual do staging em 375, 768 e 1280 px, claro e escuro	11
P-14	Descobrir o papel do banco e decidir o RLS	9.4
P-15	Subir a produção e ligar o domínio	9.4
P-16	Calendário de feriados e RLS (nesta ordem)	9.4
P-17	Primeira conta de equipe com MFA (`flask staff conceder`)	11
P-18	Webhook do Asaas em produção e cobrança real de valor baixo	9.7
P-19	Teste de isolamento A/B em produção com duas empresas	11
P-20	Backup real e teste de restauração (repetir todo mês)	9.10, 11
P-21	Monitoramento e alertas, com derrubada de teste	9.9
P-22	Onboarding do piloto e canal de suporte	10
Novo	Cadastrar os horários disponíveis para ligação em `/interno` (sem horário, o cliente não tem o que escolher)	3.4

12.3 O que eu ainda posso/devo fazer EU FAÇO

Item	Situação
Rodar a suíte completa em PostgreSQL 16 na branch principal	Pendente da sua decisão (12.1). O CI (P-11) também fará isso quando o código for enviado ao GitHub

Item	Situação
`/security-review` formal completo	Foi feita uma revisão por leitura do código novo (sem achados de alta confiança). A revisão formal do repositório inteiro não foi refeita
API: detalhe do projeto, eventos e registrar progresso (hoje "501 não implementado")	Só se você quiser a API no lançamento
Texto final de Termos e Privacidade	Depende do advogado (P-03); eu troco as páginas e removo o aviso de rascunho
Corrigir o que o smoke test e o CI acusarem no staging	Só depois de existir staging
Liberar horário reservado por dados anteriores à coluna de empresa gravada	Caso-limite registrado: só acontece com dados antigos; sem teste com RLS forçado
Telas de `/interno/horarios` e formulários logados por captura	Cobertos só por testes automáticos
Exportações (CSV/XLSX, LGPD) e JSON da API ainda trazem campos de módulos opcionais	Decisão do ADR-0017; revisar se necessário
Rolagem do "Tempo restante" em desktop e link de e-mail sem cor	Citados em pendências anteriores; não reconferidos
Pendências estruturais em `docs/plano/limpeza-pendente.md`	Fluxos de detalhe/edição do projeto sem equivalente no Preencher etapas, entre outras
Teste real com S3/R2, Resend, Sentry e Asaas	Nenhum foi exercitado com serviço real

12.4 Riscos e limites que você precisa conhecer

- **O isolamento entre empresas está provado em testes SQLite e documentação; a prova no PostgreSQL real ainda não foi refeita na branch principal.** Um agente rodou 1.172 testes em PostgreSQL local numa versão anterior, mas isso não substitui o CI nem o teste A/B em produção (P-19).
- **Banco gratuito do Render expira em 30 dias** e depois é apagado (após 14 dias de tolerância). Não use para dado real.
- **Worker e crons não têm plano gratuito**: sem eles não há alertas, relatório semanal, backup nem exclusões LGPD.
- **O papel do banco pode anular o RLS** (P-14).
- **Perder a chave privada do backup** inutiliza todos os backups; **vazá-la** expõe todos.
- **Transferência internacional de dados** (Render nos EUA): precisa estar na Política de Privacidade e aprovada pelo advogado.
- **A integração com o Asaas nunca falou com o serviço real**; a homologação pode exigir ajustes.
- **RPO de 24 horas:** com backup diário, no pior caso perde-se até um dia de lançamentos, a menos que o plano do Render ofereça recuperação ponto a ponto (confirme).
- **Dados de exemplo:** os nomes na faixa de produtos da página inicial são de exemplo.
- **Vídeos e preços:** vídeos não foram assistidos; preços em dólar mudam.

12.5 Transparência: ocorrências desta sessão que você deve saber

- **Banco de teste no Render:** minha tentativa de criar o PostgreSQL de staging foi negada pelo sistema de permissões. Nada foi criado e não contornei o bloqueio.
- **Contorno de bloqueio por um agente:** o agente do briefing, para rodar PostgreSQL local, tentou `su` / `runuser` (recusados) e usou um script próprio para trocar de usuário. Não autorizei isso; o servidor era local, vazio e sem dados reais, já estava parado, e eu apaguei os arquivos que ele deixou em `/var/tmp`.
- **Comando amplo de encerramento:** o mesmo agente usou `pkill`, que pode ter derrubado a suíte de outro agente. Por isso rodei a suíte completa na branch principal depois.
- **Envio antes do resultado:** enviei ao GitHub a branch principal antes de a suíte completa terminar (a pedido do verificador de commits pendentes). Ela passou depois: 1.114 aprovados.
- **Commit com arquivo extra:** um commit do agente das pendências levou `docs/plano/pendencias-final.md` sem mencionar na mensagem. Não reescrevi o histórico.
- **Promessa que eu não podia cumprir:** anteriormente prometi rodar a suíte em PostgreSQL sem ter acesso a um servidor. Corrigi: não faço essa afirmação sem executar.

13. Glossário

Termo	O que significa
API	Porta pela qual outro programa conversa com o sistema, sem tela.
Alembic / migração	Ferramenta e arquivos que registram, passo a passo, cada mudança na estrutura do banco. Evitam “mexer na mão” em produção.
Asaas	Plataforma brasileira de cobrança (Pix, boleto, cartão, assinaturas).
Backup	Cópia de segurança. Aqui é diária, criptografada e assinada.
Blueprint (Render)	Arquivo YAML que descreve todos os serviços para o Render criar de uma vez.
Bucket	Pasta de nível superior em um armazenamento de arquivos (como o R2).
CI	Verificação automática (no GitHub) que roda testes a cada envio de código.
Cron	Tarefa que roda sozinha em horários marcados.
CSP	Política de segurança do navegador que limita de onde o site pode carregar scripts e estilos.
CSRF	Ataque em que outro site faz o navegador do usuário enviar um pedido sem ele saber; o sistema usa um código secreto em cada formulário para barrar isso.
DKIM / SPF / DMARC	Três registros de DNS que provam que um e-mail realmente veio do seu domínio, para não cair no spam.
DNS	O “catálogo de endereços” da internet: liga o nome do site ao servidor.
Docker	Empacotamento do sistema com tudo que ele precisa, para rodar igual em qualquer lugar.
Flask	A biblioteca de Python na qual o sistema é construído.
HTMX	Biblioteca que atualiza partes da página sem recarregar tudo e sem escrever JavaScript complicado.
LGPD	Lei Geral de Proteção de Dados: regras sobre dados pessoais no Brasil.
MFA / 2FA	Segundo fator de login: além da senha, um código gerado no celular.
Módulo opcional	Função do sistema desligada por padrão que o proprietário pode ligar.
Multiempresa	Várias empresas no mesmo sistema, sem ver os dados umas das outras.
PostgreSQL	O banco de dados do sistema.
R2	Armazenamento de arquivos da Cloudflare, compatível com o S3 da Amazon.
Redis / Key Value	Memória rápida usada para limites de tentativa e filas; pode ser zerada sem perder dados importantes.
RLS	Row Level Security: trava do banco que só entrega as linhas da empresa certa.
Rollback	Voltar o sistema para a versão anterior quando um deploy dá errado.

Termo	O que significa
RPO / RTO	Quanto de dado se aceita perder (RPO, meta 24 h) e em quanto tempo o sistema volta (RTO, meta 4 h).
Sandbox	Ambiente de teste de um serviço, sem dinheiro ou dados reais.
Sentry	Serviço que recebe e organiza os erros do sistema.
Staging	Cópia de ensaio do sistema, com chaves de teste.
Token	Texto secreto que dá acesso; aqui também o nome das cores e fontes do visual (“tokens CSS”).
Webhook	Aviso automático que um serviço (como o Asaas) envia ao seu site quando algo acontece (ex.: pagamento).
Worker	Processo que roda em segundo plano para tarefas que não precisam da tela.

14. Anexos

Anexo A: onde está cada coisa no repositório

Preciso de…	Abra
Passo a passo oficial de lançamento (P-01 a P-22, smoke test, rollback, go/no-go)	`docs/plano/lancamento.md`
Deploy, e-mail, backup, monitoramento, plantão	`docs/plano/operacao.md`
Pagamentos, homologação do Asaas (15 passos)	`docs/plano/pagamentos.md`
Segurança: ameaças, cabeçalhos, limites de requisição	`docs/plano/seguranca.md`
LGPD: inventário, retenção, operadores	`docs/plano/lgpd.md`
Incidentes e rotação de segredos	`docs/plano/incidentes.md`
Decisões (ADRs 0001 a 0019)	`docs/plano/adr/`
O que cada fase entregou	`docs/plano/progresso.md`
Pendências finais	`docs/plano/pendencias-final.md` e `docs/plano/limpeza-pendente.md`
Variáveis de ambiente (modelo, sem valores)	`.env.example`
Regras e contexto para quem programa	`CLAUDE.md`

Anexo B: todos os links de vídeo, por assunto

Mesma ressalva da seção 1: encontrados por busca, não assistidos. Nenhum link foi inventado: todos vieram de resultados de busca desta sessão.

Assunto	Vídeos
Cofre de senhas e 2FA	Bitwarden: 2 Step Verification https://www.youtube.com/watch?v=V5uge64ZN7U Bitwarden: 2FA guia 2026 https://www.youtube.com/watch?v=qHU1rUfQSb0
Domínio (Registro.br)	Registrar domínio, atualizado 2026 https://www.youtube.com/watch?v=zPXlVWTAFGU .com.br em 3 minutos https://www.youtube.com/watch?v=jZmquG4fmNQ Registrar e configurar DNS https://www.youtube.com/watch?v=WhtAQVTb_DI Apontamento de DNS https://www.youtube.com/watch?v=TCCYDZaRJ80

Assunto	Vídeos
Cloudflare DNS	Add Nameservers 2026 https://www.youtube.com/watch?v=yEEnyOKiZF0 Change Nameservers, full guide https://www.youtube.com/watch?v=tPLLPdFLQ68 Add Domain to Cloudflare https://www.youtube.com/watch?v=uTbWRgCSt18 Point Your Domain to Cloudflare https://www.youtube.com/watch?v=XgfRHdFjD_Y
Cloudflare R2	Bucket e domínio https://www.youtube.com/watch?v=ZvczexjXt7E S3 API compatibility https://www.youtube.com/watch?v=ohfg-ICt6hc Upload 2026 https://www.youtube.com/watch?v=pQmGBSRTdPQ
Render + Flask + PostgreSQL	Flask e Postgres no Render https://www.youtube.com/watch?v=IBfj_OZf2Mo Flask e Postgres no Render, grátis https://www.youtube.com/watch?v=iH0XIesfwu4 Docker + PostgreSQL no Render https://www.youtube.com/watch?v=_IdejOUafe4
Resend e e-mail (SPF, DKIM, DMARC)	Verify a Domain in Resend https://www.youtube.com/watch?v=CF6gRy-9_LE Resend: API e verificação de domínio https://www.youtube.com/watch?v=pHY8nzgfz4M Getting Started with Resend https://www.youtube.com/watch?v=FzoakVwRm2Q DNS de e-mail no Registro.br https://www.youtube.com/watch?v=vR9MrAMCbqM O que é e como configurar o DMARC https://www.youtube.com/watch?v=PufbDpElnpc
Asaas	Sandbox https://www.youtube.com/watch?v=VSfi3dNI9RY Webhooks https://www.youtube.com/watch?v=Yk6yTfaLVDY Assinaturas https://www.youtube.com/watch?v=m-ahYNfK_UU Fila de webhooks https://www.youtube.com/watch?v=q3hZQzLx4XM
Sentry	Monitorar erros com o Sentry (PT) https://www.youtube.com/watch?v=5OTXem5LFTI Sentry com Flask https://www.youtube.com/watch?v=zVW83QUSDdQ
Monitor de uptime	UptimeRobot (PT) https://www.youtube.com/watch?v=ofXuyWbt9aY Monitorando com UptimeRobot (PT) https://www.youtube.com/watch?v=DszgYLBhrs0

Assunto	Vídeos
LGPD	LGPD no SaaS https://www.youtube.com/watch?v=5cDydr-rjzg Impactos da LGPD no desenvolvimento https://www.youtube.com/watch?v=xb5WS47nmCY LGPD em 4 minutos https://www.youtube.com/watch?v=n3e0HVcNmlO
RLS (multiempresa no banco)	RLS Series Part 1 https://www.youtube.com/watch?v=8qPTZQvJ9fA RLS para SaaS multiempresa https://www.youtube.com/watch?v=DabJ0hxiXAo Row Level Security Policy https://www.youtube.com/watch?v=98yxrZ3Aip4
Migrações (Alembic)	Flask-Migrate https://www.youtube.com/watch?v=uNmWxvvyBGU Migrations com Flask-Migrate https://www.youtube.com/watch?v=_aDEn4n32uY Alembic https://www.youtube.com/watch?v=bfelC61XKO4
Backup do PostgreSQL	pg_dump (PT) https://www.youtube.com/watch?v=ZBWx-CA0jdk pg_dump e pg_restore https://www.youtube.com/watch?v=WB6WzuFHcP8 Dump e restore https://www.youtube.com/watch?v=bWtsdj68JBo
Segurança web	CSP contra XSS https://www.youtube.com/watch?v=B0Rz_qiQAWo CSRF (OWASP) https://www.youtube.com/watch?v=gWMoj9FYTj4 XSS (OWASP) https://www.youtube.com/watch?v=u1rxapgQA2A
HTMX com Flask	Projeto Flask HTMX (PT) https://www.youtube.com/watch?v=OmPmH_3xLo8 HTMX + Flask https://www.youtube.com/watch?v=kNXVFB5eQfo CRUD com HTMX e Flask https://www.youtube.com/live/O2Xd6DmcB9g

Anexo C: fontes citadas sobre os planos do Render

- Deploy for Free
- Free PostgreSQL instances now expire after 30 days
- Services and Service Types
- All New Free Instance Types On Render
- Cron jobs vs background workers

São páginas da própria Render; os valores em dólar podem mudar.

Fim do guia

Ao concluir cada item da seção 12.2, me avise o resultado (sem enviar chaves). Eu confiro o que consigo ver e ajusto o código quando um teste real acusar algo.